

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_d9866476-800\agent-a0b8009aeba7d0373.jsonl`  
- SHA-256: `168fb230aaccb92ff6dc1e0fc473a2d24863e26b699f82a59f9d6f75fd281bee`  
- Source modified: `2026-06-22T09:11:35+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_d9866476-800`  
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`
```

Transcript

1. user (2026-06-22T09:08:03.306Z)

You are mapping the badminton-highlight-indexer repo, checked out at origin/master under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e. Read files under that ABSOLUTE path (Read/Grep/Glob). Be concrete: cite file:line / file::symbol. The goal is to design a NIGHTLY regression that runs the CONFIGURED + CHECKPOINTED pipeline on 2-3 short golden videos and asserts the NUMBER OF REELS (highlight clips) does not change. Return raw structured data, not prose for a human.

INTEGRATION with the EXISTING nightly: read scripts/nightly_regression.py, scripts/nightly_regression.ps1, scripts/register_nightly_task.ps1, eval_baselines/nightly_baseline.json (all just merged in PR #202 ? a CPU re-score of CACHED trajectories, NOT a full-pipeline reel run). Describe the established pattern (manifest/config-list driven, frozen baseline JSON, dated output/<dir>/<date>.md report, exit codes 0/1/4, scheduled-task wrapper) so the NEW full-pipeline reel-count test REUSES it. Recommend: extend nightly_regression.py vs a sibling scripts/nightly_reelcount.py. Note how to make adding more videos a one-line manifest edit (the extensible pattern).

2. assistant (2026-06-22T09:08:06.181Z)

I'll explore the badminton-highlight-indexer repository to understand the nightly regression infrastructure and design a new full-pipeline reel-count test.

3. user (2026-06-22T09:08:06.928Z)

```
689c738 docs(pkg): mark P2c ? + the loop closed ? v1 LIGHT is feature-complete (#313)  
293e5c0 feat(pkg): re-bundle backend/ui ? serve the P2c first-run wizard from indexer --app  
(#312)  
f8b2c3b feat(pkg): P2b-install ? batteries-included install + truly-local 'motion' first-run  
default (gate 08) (#310)  
0ed4cc6 fix(segmenter): fail the job on an unrunnable config instead of a silent 0-rally success  
(Finding B) (#311)  
a42c132 docs(pkg): track the batteries-included app plan (PACKAGING_PLAN.md) + index pointers  
(#309)  
de2bf14 feat(pkg): P2e ? lazy-torch so the LIGHT tier imports with NO torch installed (default-  
OFF) (#308)  
679e642 feat(pkg): P2d ? DB upgrade-safety (downgrade guard + backup-before-migrate + export)  
(default-OFF) (#307)  
f01dc85 feat(db): v7 ? record the UI locale per job (PMF analytics) + count_jobs_by_locale  
(#306)  
fa7ee3f feat(pkg): P2a ? single ffmpeg/ffprobe resolver (env-overridable; one fail-loud place)  
(default-OFF) (#305)  
e99d0c3 feat(stitch): localize the reel watermark to the UI locale (+ OFL Noto fonts) (#304)
```

4. assistant (2026-06-22T09:08:09.703Z)

Let me read the key nightly regression files:

5. user (2026-06-22T09:08:11.697Z)

```
1      ""Nightly rally-quality regression tripwire.
2
3      Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
4      seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
diffs
5      the headline metrics against a FROZEN baseline
(``eval_baselines/nightly_baseline.json``).
6      Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-
video
7      over-segmentation-guard regression.
8
9      This is the owner's dual-eval-set discipline made automatic (``memory/eval-dual-set-
regression``
10     + ``docs/R2_EVAL_METRIC_DESIGN.md`` §4.5): the growing GOLDEN-finalized set is re-run
every
11     night so a *late* change cannot silently lose an *earlier* win. It is **default-
ADDITIVE** ? it
12     does NOT change the promotion gate. It is a *tripwire*, not a re-training run: no GPU,
no Gemini,
13     no WASB re-extraction (those are cached and unchanged night to night).
14
15     Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base
knobs):
16
17     * **DEFAULT** ? the SERVED production windowing, i.e. what users get today. Catches a
regression
18     to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
flipped ON, so
19     SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
20     * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
21     ``docs/QUALITY_ITERATIONS.md`` R3/R4). Now == DEFAULT (production shipped it); kept
explicit so
22     the nightly still names the milestone knobs and stays meaningful if production later
diverges.
23     * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
The
24     permanent historical floor, so each report keeps showing the delta the milestone buys
(a launched
25     win must keep standing its ground, not be taken at launch-time value).
26
27     Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
trajectory CSVs
28     outside the repo + ``output/<name>_golden_features.json``); the corpus is NOT committed.
So the
29     baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
baseline``
30     whenever the corpus grows or an intended improvement lands. (A cloud agent does not
suffice ? it
31     has the repo but not the cached trajectories; hence the local Windows Scheduled Task
wiring in
32     ``scripts/nightly_regression.ps1``.)
33
34     Usage
35     ----
36     Snapshot the baseline from current HEAD (after an intended improvement or a corpus
change):
37
38     python scripts/nightly_regression.py --update-baseline
39
40     Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
``.json``):
```

```

41
42     python scripts/nightly_regression.py
43
44     Exit codes
45     -----
46     * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
47     * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
per-video
48     over-seg guard increased). Takes precedence over a stale baseline.
49     * ``2`` ? operational error (manifest/corpus missing, nothing scored).
50     * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
51     * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
needed,
52     but no regression was found on the comparable intersection.
53     """
54     from __future__ import annotations
55
56     import argparse
57     import datetime as _dt
58     import json
59     import os
60     import sys
61     from dataclasses import dataclass, field, replace
62     from typing import Any, Dict, List, Optional, Tuple
63
64     # ----- #
65     # Paths (resolved relative to the repo root, so the script is cwd-robust).
66     # ----- #
67     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
68     if REPO_ROOT not in sys.path:
69         sys.path.insert(0, REPO_ROOT) # importable as `python
scripts/nightly_regression.py` from anywhere
70     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
71     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
72     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
73     DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
74     DEFAULT_IOU = 0.5
75
76     # ----- #
77     # What we gate on (a tripwire ? focused, not every metric).
78     # * HIGHER-better aggregates regress when they DROP beyond ``metric_tol``.
79     # * merge_rate (lower-better) regresses when it RISES beyond ``rate_tol``.
80     # * per-video split_count / merge_count must NOT increase beyond ``over_seg_tol``
81     # (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
82     # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
gated.
83     # ----- #
84     GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
85     GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
86     PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
87
88     #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
context).
89     AGG_SNAPSHOT: Tuple[str, ...] = (
90         "tol_f1", "tol_precision", "tol_recall", "f1@0.5", "f1@0.25", "precision", "recall",
91         "merge_rate", "split_rate", "segment_count_ratio", "dstart_abs_median",
"dend_abs_median",
92     )
93     #: Per-video metrics snapshotted into the baseline.
94     PV_SNAPSHOT: Tuple[str, ...] = (
95         "tol_f1", "f1@0.5", "f1@0.25", "merge_rate", "split_count", "merge_count",
"num_pred", "num_gt",
96     )
97
98     DEFAULT_METRIC_TOL = 0.005 # F1-like drop that counts as a regression (absorbs float

```

```

jitter)
99     DEFAULT_RATE_TOL = 0.010      # merge_rate rise that counts as a regression
100    DEFAULT_OVER_SEG_TOL = 0      # per-video split/merge increase allowed (strict)
101
102
103    # ----- #
104    # The two tracked configs.
105    # ----- #
106    @dataclass(frozen=True)
107    class NightlyConfig:
108        """A named windowing config the nightly re-scores (preset + net-crossings gate)."""
109
110        name: str
111        preset: Any          # backend.eval.windowing.WindowingPreset
112        min_crossings: int = 0
113        note: str = ""
114
115
116    def tracked_configs() -> List[NightlyConfig]:
117        """DEFAULT (SERVED = shipped) + MILESTONE (explicit cap=6 + R4) + BASELINE_OFF (the
pre-flip floor).
118
119        All three are anchored on the SAME ``windowing.SERVED`` base knobs (velocity_thresh
/ merge_gap /
120        min_window) so they differ ONLY in the quality knobs ? they are a clean A/B/C:
121
122        * **DEFAULT** ? ``SERVED`` verbatim, so it can never silently drift from the served
operating
123        point. **Post-#204 (2026-06-18) the R1 milestone is flipped ON ? SERVED IS cap=6 +
R4, so
124        DEFAULT now equals MILESTONE.** Its job: catch an *accidental* change to a
production windowing
125        knob.
126        * **MILESTONE** ? SERVED + the explicit R3/R4 knobs (``docs/QUALITY_ITERATIONS.md``
R3/R4). Pins
127        the milestone *definition* independent of what production ships, so "is cap=6 + R4
still scoring
128        what we expect?" stays answerable even if a future change moves DEFAULT.
129        * **BASELINE_OFF** ? SERVED with every quality knob forced OFF (the pre-R1-flip
windowing). Pins
130        the *historical floor* so each nightly report keeps showing the delta the
milestone buys (the
131        dual-eval-set discipline: a launched win must keep standing its ground, not be
taken at
132        launch-time value). It is the reference DEFAULT used to be before the flip."""
133        from backend.eval.windowing import SERVED
134
135        default = NightlyConfig(
136            name="default", preset=SERVED, min_crossings=0,
137            note="SERVED production windowing ? now the R1 milestone (cap=6 + R4) after the
#204 flip.",
138        )
139        milestone = NightlyConfig(
140            name="milestone",
141            preset=replace(SERVED, name="milestone", max_window_duration=6.0,
142                          inactivity_end=1.5, inactivity_rally_thresh=0.20,
143                          inactivity_min_density=0.30),
144            min_crossings=0,
145            note="R3 cap=6 s + R4 rally-END inactivity trim ? the shipped milestone (==
default post-#204).",
146        )
147        baseline_off = NightlyConfig(
148            name="baseline_off",
149            preset=replace(SERVED, name="baseline_off", max_window_duration=0.0,
hard_cap=False,

```

```

150             inactivity_end=0.0, inactivity_rally_thresh=0.08,
inactivity_min_density=0.34,
151             blob_fallback=False, blob_factor=4.0),
152             min_crossings=0,
153             note="Pre-R1-flip all-OFF windowing ? the historical floor; pins the value the
milestone buys.",
154         )
155         return [default, milestone, baseline_off]
156
157
158     # ----- #
159     # Tolerances bundle.
160     # ----- #
161     @dataclass(frozen=True)
162     class Tolerances:
163         metric_tol: float = DEFAULT_METRIC_TOL
164         rate_tol: float = DEFAULT_RATE_TOL
165         over_seg_tol: int = DEFAULT_OVER_SEG_TOL
166
167         def to_dict(self) -> Dict[str, Any]:
168             return {"metric_tol": self.metric_tol, "rate_tol": self.rate_tol,
169                     "over_seg_tol": self.over_seg_tol}
170
171
172     # ----- #
173     # Pure summary + comparison core (no I/O ? unit-testable).
174     # ----- #
175     def summarize_run(eval_result: Dict[str, Any]) -> Dict[str, Any]:
176         """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.
177
178         Keeps only the deterministic quantities we compare ? aggregate MEANS (not the
bootstrap CI,
179         which is seeded but irrelevant to the gate) and per-video RAW metric values."""
180         agg = eval_result.get("aggregate", {}) or {}
181         aggregate = {k: (agg[k]["mean"] if isinstance(agg.get(k), dict) else None) for k in
AGG_SNAPSHOT}
182         per_video = {r["name"]: {k: r[k] for k in PV_SNAPSHOT if k in r} for r in
eval_result.get("rows", [])}
183         return {
184             "n": int(agg.get("n", len(eval_result.get("rows", [])))),
185             "preset": eval_result.get("preset"),
186             "min_crossings": eval_result.get("min_crossings", 0),
187             "iou": eval_result.get("iou", DEFAULT_IOU),
188             "aggregate": aggregate,
189             "per_video": per_video,
190         }
191
192
193     @dataclass
194     class Finding:
195         """One comparison outcome line (regression / improvement / stale note)."""
196
197         config: str
198         scope: str          # "aggregate" | "per_video" | "config"
199         metric: str
200         kind: str           # "regression" | "improvement" | "stale"
201         baseline: Optional[float] = None
202         current: Optional[float] = None
203         video: Optional[str] = None
204
205         @property
206         def delta(self) -> Optional[float]:
207             if self.baseline is None or self.current is None:
208                 return None
209             return self.current - self.baseline

```

```

210
211     def message(self) -> str:
212         loc = f"{self.config}/{self.video}/{self.metric}" if self.video else
213         f"{self.config}/{self.metric}"
214         if self.kind == "stale":
215             return f"[STALE] {self.config}: {self.metric}"
216         d = self.delta
217         ds = f"{d:+.4f}" if d is not None else "?"
218         b = "?" if self.baseline is None else f"{self.baseline:.4f}"
219         c = "?" if self.current is None else f"{self.current:.4f}"
220         tag = "REGRESSION" if self.kind == "regression" else "improvement"
221         return f"[{tag}] {loc}: {b} ? {c} ({ds})"
222
223     @dataclass
224     class ConfigComparison:
225         config: str
226         status: str = "ok" # "ok" | "regression" | "stale"
227         findings: List[Finding] = field(default_factory=list)
228         added_videos: List[str] = field(default_factory=list)
229         removed_videos: List[str] = field(default_factory=list)
230         aggregate_gated: bool = True # False when preset/corpus changed ? aggregate not
231         comparable
232         @property
233         def regressions(self) -> List[Finding]:
234             return [f for f in self.findings if f.kind == "regression"]
235
236         @property
237         def improvements(self) -> List[Finding]:
238             return [f for f in self.findings if f.kind == "improvement"]
239
240
241     def _preset_comparable(base_preset: Any, cur_preset: Any) -> bool:
242         """Are the two stored preset dicts the same windowing definition? (name is
243         cosmetic)."""
244         if not isinstance(base_preset, dict) or not isinstance(cur_preset, dict):
245             return base_preset == cur_preset
246         keys = set(base_preset) | set(cur_preset)
247         return all(base_preset.get(k) == cur_preset.get(k) for k in keys if k != "name")
248
249     def compare_config(name: str, base_sum: Dict[str, Any], cur_sum: Dict[str, Any],
250                       tols: Tolerances) -> ConfigComparison:
251         """Compare one config's current summary to its baseline summary.
252
253         * If the preset definition or the net-crossings gate changed, the whole config is
254         STALE
255         (numbers aren't comparable) ? flag it, skip gating.
256         * If the corpus (video set) changed, the aggregate is not comparable (different
257         videos) ?
258         skip the aggregate gate + flag added/removed, but STILL run the per-video gate on
259         the
260         INTERSECTION (a code regression on a shared video must still trip the wire).
261         """
262         cmp = ConfigComparison(config=name)
263
264         # 1) Config-definition drift ? stale, not gateable.
265         if not _preset_comparable(base_sum.get("preset"), cur_sum.get("preset")) \
266             or base_sum.get("min_crossings") != cur_sum.get("min_crossings"):
267             cmp.status = "stale"
268             cmp.aggregate_gated = False
269             cmp.findings.append(Finding(config=name, scope="config", metric="preset",
270                                       kind="stale"))
271         return cmp

```

```

268
269     base_pv = base_sum.get("per_video", {})
270     cur_pv = cur_sum.get("per_video", {})
271     base_names, cur_names = set(base_pv), set(cur_pv)
272     cmp.added_videos = sorted(cur_names - base_names)
273     cmp.removed_videos = sorted(base_names - cur_names)
274     corpus_changed = bool(cmp.added_videos or cmp.removed_videos)
275
276     # 2) Aggregate gate (only when the corpus matches ? means over different sets aren't
comparable).
277     if corpus_changed:
278         cmp.aggregate_gated = False
279         cmp.status = "stale"
280         cmp.findings.append(Finding(config=name, scope="config", metric="corpus",
kind="stale"))
281     else:
282         base_agg = base_sum.get("aggregate", {})
283         cur_agg = cur_sum.get("aggregate", {})
284         for m in GATED_HIGHER:
285             b, c = base_agg.get(m), cur_agg.get(m)
286             if b is None or c is None:
287                 continue
288             if c < b - tols.metric_tol:
289                 cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
290             elif c > b + tols.metric_tol:
291                 cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
292         for m in GATED_LOWER:
293             b, c = base_agg.get(m), cur_agg.get(m)
294             if b is None or c is None:
295                 continue
296             if c > b + tols.rate_tol:
297                 cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
298             elif c < b - tols.rate_tol:
299                 cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
300
301     # 3) Per-video over-seg guard on the intersection (always ? catches per-video code
regressions).
302     for vid in sorted(base_names & cur_names):
303         for m in PER_VIDEO_GUARD:
304             b, c = base_pv[vid].get(m), cur_pv[vid].get(m)
305             if b is None or c is None:
306                 continue
307             if c > b + tols.over_seg_tol:
308                 cmp.findings.append(Finding(name, "per_video", m, "regression",
float(b), float(c), video=vid))
309
310     if cmp.regressions:
311         cmp.status = "regression"
312     elif cmp.status != "stale":
313         cmp.status = "ok"
314     return cmp
315
316
317 @dataclass
318 class NightlyResult:
319     comparisons: List[ConfigComparison] = field(default_factory=list)
320     skipped: Dict[str, List[str]] = field(default_factory=dict) # config -> manifest
videos not scored
321
322     @property
323     def has_regression(self) -> bool:
324         return any(c.status == "regression" for c in self.comparisons)
325
326     @property
327     def has_stale(self) -> bool:

```

```

328         return any(c.status == "stale" for c in self.comparisons)
329
330     def overall_status(self) -> str:
331         if self.has_regression:
332             return "REGRESSION"
333         if self.has_stale:
334             return "STALE"
335         return "PASS"
336
337     def exit_code(self) -> int:
338         if self.has_regression:
339             return 1
340         if self.has_stale:
341             return 4
342         return 0
343
344
345     def compare_all(baseline: Dict[str, Any], current: Dict[str, Any],
346                    tols: Tolerances, skipped: Optional[Dict[str, List[str]]] = None) ->
NightlyResult:
347         """Compare every tracked config's current summary to the baseline summary."""
348         res = NightlyResult(skipped=skipped or {})
349         base_cfgs = baseline.get("configs", {})
350         cur_cfgs = current.get("configs", {})
351         for name in cur_cfgs:
352             if name not in base_cfgs:
353                 cmp = ConfigComparison(config=name, status="stale", aggregate_gated=False)
354                 cmp.findings.append(Finding(config=name, scope="config",
metric="missing_in_baseline", kind="stale"))
355                 res.comparisons.append(cmp)
356                 continue
357             res.comparisons.append(compare_config(name, base_cfgs[name], cur_cfgs[name],
tols))
358         return res
359
360
361     # ----- #
362     # Report formatting (pure).
363     # ----- #
364     def _fmt(v: Optional[float]) -> str:
365         return "?" if v is None else f"{v:.4f}"
366
367
368     def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
369                      result: NightlyResult, tols: Tolerances, date: str) -> str:
370         L: List[str] = []
371         status = result.overall_status()
372         badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION", "STALE": "? STALE (baseline
refresh needed)"}[status]
373         L.append(f"# Nightly rally-quality regression ? {date}")
374         L.append("")
375         L.append(f"**Status: {badge}** . exit code {result.exit_code()}")
376         L.append("")
377         L.append(f"- HEAD: `{current.get('git_commit', '?')}` . baseline: "
378                 f"`{baseline.get('git_commit', '?')}` (snapshot
{baseline.get('generated_at', '?')})")
379         L.append(f"- Corpus manifest: `{current.get('manifest', '?')}` .
IoU={current.get('iou', DEFAULT_IOU)}")
380         L.append(f"- Tolerances: metric  $\pm$ {tols.metric_tol}, rate  $\pm$ {tols.rate_tol}, "
381                 f"per-video over-seg  $\pm$ {tols.over_seg_tol}")
382         L.append("")
383
384         # Up-front regression / stale summary.
385         regs = [f for c in result.comparisons for f in c.regressions]
386         if regs:

```



```

387         L.append("## ? Regressions")
388         L.append("")
389         for f in regs:
390             L.append(f"- {f.message()}")
391         L.append("")
392     if result.has_stale:
393         L.append("## ? Stale / not comparable")
394         L.append("")
395         for c in result.comparisons:
396             if c.status == "stale":
397                 if c.added_videos:
398                     L.append(f"- {c.config}`: corpus added {c.added_videos}")
399                 if c.removed_videos:
400                     L.append(f"- {c.config}`: corpus removed {c.removed_videos}")
401                 if not c.added_videos and not c.removed_videos:
402                     L.append(f"- {c.config}`: config definition changed vs baseline ?
re-snapshot with --update-baseline")
403                 L.append("- Action: `python scripts/nightly_regression.py --update-baseline`
(after confirming the change is intended).")
404                 L.append("")
405
406         # Skipped (no silent caps).
407         skipped_any = {k: v for k, v in result.skipped.items() if v}
408         if skipped_any:
409             L.append("## ? Skipped manifest videos (missing trajectory or GT ? NOT silently
dropped)")
410             L.append("")
411             for cfg, vids in skipped_any.items():
412                 L.append(f"- {cfg}`: {vids}")
413             L.append("")
414
415         # Per-config detail.
416         cur_cfgs = current.get("configs", {})
417         base_cfgs = baseline.get("configs", {})
418         for cmp in result.comparisons:
419             cur = cur_cfgs.get(cmp.config, {})
420             base = base_cfgs.get(cmp.config, {})
421             L.append(f"## Config `{cmp.config}` ? {cur.get('note', '')}")
422             L.append("")
423             L.append(f"_status: {cmp.status}; n={cur.get('n', '?')} (baseline
n={base.get('n', '?')})")
424             L.append("")
425             cagg = cur.get("aggregate", {})
426             bagg = base.get("aggregate", {})
427             L.append("| metric | baseline | current | ? | gated |")
428             L.append("| ---|---|---|---|---|")
429             for m in AGG_SNAPSHOT:
430                 b, c = bagg.get(m), cagg.get(m)
431                 d = (c - b) if (b is not None and c is not None) else None
432                 gated = "?" if (m in GATED_HIGHER or m in GATED_LOWER) and
cmp.aggregate_gated else ""
433                 L.append(f"| {m} | {fmt(b)} | {fmt(c)} | {fmt(d)} | {gated} |")
434             L.append("")
435             # Per-video over-seg guard table.
436             cpv = cur.get("per_video", {})
437             bpv = base.get("per_video", {})
438             names = sorted(set(cpv) | set(bpv))
439             if names:
440                 L.append("| video | tolF1 (b?c) | split (b?c) | merge (b?c) |")
441                 L.append("| ---|---|---|---|")
442                 for v in names:
443                     cb, cc = bpv.get(v, {}), cpv.get(v, {})
444                     tf = f"{fmt(cb.get('tol_f1'))}?{fmt(cc.get('tol_f1'))}"
445                     sp = f"{cb.get('split_count', '?')}?{cc.get('split_count', '?')}"
446                     mg = f"{cb.get('merge_count', '?')}?{cc.get('merge_count', '?')}"

```

```

447         flag = " ?" if any(f.video == v and f.kind == "regression" for f in
cmp.findings) else ""
448         L.append(f"| {v}{flag} | {tf} | {sp} | {mg} |")
449         L.append("")
450         imps = cmp.improvements
451         if imps:
452             L.append("_Improvements over baseline (consider `--update-baseline` to
ratchet the floor up):_")
453             for f in imps:
454                 L.append(f"- {f.message()}")
455             L.append("")
456
457         L.append("---")
458         L.append("_Tripwire only ? re-scores CACHED trajectories (no GPU/Gemini/WASB). Does
not change "
459             "the promotion gate. See docs/R2_EVAL_METRIC_DESIGN.md + memory/eval-dual-
set-regression._")
460         return "\n".join(L)
461
462
463     # ----- #
464     # I/O shell.
465     # ----- #
466     def _git_commit() -> str:
467         import subprocess
468         try:
469             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
capture_output=True, text=True, timeout=10)
470             return out.stdout.strip() or "?"
471         except Exception:
472             return "?"
473
474
475
476     def _display_path(path: str) -> str:
477         """A clean, portable form for the committed baseline: ``output/<file>`` when the
path lives
478         in an ``output`` dir (the production layout), else repo-relative, else the path as
given.
479         Metadata only ? the comparison never reads it."""
480         norm = path.replace("\\", "/")
481         parent = os.path.basename(os.path.dirname(norm))
482         if parent == "output":
483             return "output/" + os.path.basename(norm)
484         try:
485             rel = os.path.relpath(path, REPO_ROOT)
486             if not rel.startswith(".."):
487                 return rel.replace("\\", "/")
488         except ValueError:
489             pass
490         return norm
491
492
493     def run_corpus(configs: List[NightlyConfig], manifest: str, features_dir: str,
494                   iou: float = DEFAULT_IOU) -> Tuple[Dict[str, Any], Dict[str, List[str]]]:
495         """Score every tracked config over the golden corpus. Returns (snapshot, skipped-
per-config).
496
497         ``snapshot`` is the serialisable structure stored as / compared to the baseline.
498         ``skipped``
499         lists manifest videos NOT scored (missing trajectory or empty GT) per config ?
500         surfaced in
501         the report so the corpus can't silently shrink."""
502         from backend.eval import rally_seg_eval
503
504         golden = rally_seg_eval.load_golden(manifest, features_dir)

```

```

503     manifest_names = [v["name"] for v in golden]
504     if not golden:
505         raise RuntimeError(f"no golden videos loaded from {manifest}")
506
507     snapshot: Dict[str, Any] = {"manifest": _display_path(manifest), "iou": iou,
"configs": {}}
508     skipped: Dict[str, List[str]] = {}
509     scored_any = False
510     for cfg in configs:
511         result = rally_seg_eval.evaluate(golden, cfg.preset, iou=iou,
min_crossings=cfg.min_crossings)
512         summary = summarize_run(result)
513         summary["note"] = cfg.note
514         snapshot["configs"][cfg.name] = summary
515         scored = set(summary["per_video"])
516         skipped[cfg.name] = [n for n in manifest_names if n not in scored]
517         scored_any = scored_any or bool(scored)
518     if not scored_any:
519         raise RuntimeError(
520             f"scored 0 videos for every config ? are the trajectory CSVs in {manifest}
present "
521             f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?"
522         return snapshot, skipped
523
524
525     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
526         if not os.path.exists(path):
527             return None
528         with open(path, encoding="utf-8") as fh:
529             return json.load(fh)
530
531
532     def save_baseline(path: str, snapshot: Dict[str, Any], tols: Tolerances) -> None:
533         out = dict(snapshot)
534         out["generated_at"] = _dt.date.today().isoformat()
535         out["git_commit"] = _git_commit()
536         out["tolerances"] = tols.to_dict()
537         out["_doc"] = ("Frozen nightly rally-quality regression baseline. Regenerate with "
538             "`python scripts/nightly_regression.py --update-baseline` after an
intended "
539             "`improvement or a corpus change. Tied to the LOCAL golden corpus (not
committed).")
540         os.makedirs(os.path.dirname(path), exist_ok=True)
541         tmp = path + ".tmp"
542         with open(tmp, "w", encoding="utf-8") as fh:
543             json.dump(out, fh, indent=2, sort_keys=True)
544             fh.write("\n")
545         os.replace(tmp, path)
546
547
548     def build_parser() -> argparse.ArgumentParser:
549         p = argparse.ArgumentParser(description="Nightly rally-quality regression tripwire
(offline, CPU).")
550         p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
551         p.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR,
552             help="dir holding <name>_golden_features.json")
553         p.add_argument("--baseline", default=DEFAULT_BASELINE, help="frozen baseline JSON
path")
554         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR, help="where dated reports
are written")
555         p.add_argument("--iou", type=float, default=DEFAULT_IOU)
556         p.add_argument("--metric-tol", type=float, default=DEFAULT_METRIC_TOL,
557             help="F1-like aggregate drop that counts as a regression")
558         p.add_argument("--rate-tol", type=float, default=DEFAULT_RATE_TOL,
559             help="merge_rate rise that counts as a regression")

```

```

560         p.add_argument("--over-seg-tol", type=int, default=DEFAULT_OVER_SEG_TOL,
561                         help="per-video split/merge-count increase allowed before it's a
regression")
562         p.add_argument("--update-baseline", action="store_true",
563                         help="snapshot the current numbers as the new baseline instead of
checking")
564         p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
565         p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
566         return p
567
568
569     def main(argv: Optional[List[str]] = None) -> int:
570         import logging
571         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
572         log = logging.getLogger("nightly_regression")
573         args = build_parser().parse_args(argv)
574         tols = Tolerances(metric_tol=args.metric_tol, rate_tol=args.rate_tol,
over_seg_tol=args.over_seg_tol)
575         date = args.date or _dt.date.today().isoformat()
576
577         if not os.path.exists(args.manifest):
578             log.error("manifest not found: %s ? the golden corpus must be present locally.",
args.manifest)
579             return 2
580
581         try:
582             snapshot, skipped = run_corpus(tracked_configs(), args.manifest,
args.features_dir, args.iou)
583         except RuntimeError as e:
584             log.error("%s", e)
585             return 2
586         snapshot["git_commit"] = _git_commit()
587         snapshot["generated_at"] = date
588
589         if args.update_baseline:
590             save_baseline(args.baseline, snapshot, tols)
591             log.info("baseline snapshot written to %s", args.baseline)
592             for cfg, s in snapshot["configs"].items():
593                 agg = s["aggregate"]
594                 log.info(" %s: n=%s tolF1=%.3f f1@0.5=%.3f merge_rate=%.3f",
595                         cfg, s["n"], agg.get("tol_f1") or 0.0, agg.get("f1@0.5") or 0.0,
596                         agg.get("merge_rate") or 0.0)
597             return 0
598
599         baseline = load_baseline(args.baseline)
600         if baseline is None:
601             log.error("no baseline at %s ? snapshot one with --update-baseline (then commit
it).", args.baseline)
602             return 3
603
604         result = compare_all(baseline, snapshot, tols, skipped)
605         report = format_report(baseline, snapshot, result, tols, date)
606
607         if not args.no_report:
608             os.makedirs(args.report_dir, exist_ok=True)
609             md_path = os.path.join(args.report_dir, f"{date}.md")
610             json_path = os.path.join(args.report_dir, f"{date}.json")
611             with open(md_path, "w", encoding="utf-8") as fh:
612                 fh.write(report + "\n")
613             with open(json_path, "w", encoding="utf-8") as fh:
614                 json.dump({"date": date, "status": result.overall_status(),
615                         "exit_code": result.exit_code(), "current": snapshot,
616                         "regressions": [f.message() for c in result.comparisons for f in

```

```

c.regressions]],
617             fh, indent=2, sort_keys=True)
618         log.info("report written to %s", md_path)
619
620         print(report)
621         code = result.exit_code()
622         if code == 1:
623             log.error("NIGHTLY REGRESSION DETECTED (%d) ? see %s",
624                       sum(len(c.regressions) for c in result.comparisons), args.report_dir)
625         elif code == 4:
626             log.warning("baseline STALE ? corpus/config changed; refresh with --update-
baseline")
627         else:
628             log.info("nightly regression check PASSED")
629         return code
630
631
632     if __name__ == "__main__":
633         raise SystemExit(main())
634

```

6. user (2026-06-22T09:08:11.828Z)

```

1      <#
2      .SYNOPSIS
3          Nightly rally-quality regression tripwire ? wrapper for the Windows Scheduled Task.
4
5      .DESCRIPTION
6          Re-scores the golden corpus under the current checkout via
scripts/nightly_regression.py
7          (CPU, seconds ? the WASB trajectories are CACHED) and diffs vs the frozen baseline.
Writes a
8          dated report + log under output/nightly_regression/ and a LATEST_STATUS.txt marker,
then exits
9          with the checker's exit code so Task Scheduler records PASS / REGRESSION / STALE.
10
11         GRACEFUL no-op (exit 0) when the checker or the local golden corpus isn't present
yet (e.g.
12         before this PR is merged into the checkout the task points at) ? so a registered
task never
13         errors in the interim and self-activates once the files land.
14
15         Cloud agents can't do this: the cached trajectories + golden features live OUTSIDE
the repo on
16         this machine. Hence a LOCAL scheduled task, not a CronCreate/cloud routine.
17
18     .PARAMETER RepoRoot
19         Repo root to test. Default: the parent of this script's directory.
20
21     .PARAMETER PythonExe
22         Python interpreter. Default: 'python'.
23
24     .PARAMETER Pull
25         If set, 'git pull --ff-only' before scoring (off by default ? never auto-touch a
shared checkout
26         that may hold a sibling session's WIP).
27     #>
28     [CmdletBinding()]
29     param(
30         [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
31         [string]$PythonExe = 'python',
32         [switch]$Pull
33     )
34

```

```

35     $ErrorActionPreference = 'Continue'
36     $date = Get-Date -Format 'yyyy-MM-dd'
37     $reportDir = Join-Path $RepoRoot 'output\nightly_regression'
38     $logDir = Join-Path $reportDir 'logs'
39     New-Item -ItemType Directory -Force -Path $logDir | Out-Null
40     $log = Join-Path $logDir "$date.log"
41     $statusFile = Join-Path $reportDir 'LATEST_STATUS.txt'
42
43     function Write-Status([string]$status, [int]$code, [string]$detail) {
44         $stamp = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
45         "$stamp status=$status exit=$code $detail" | Out-File -FilePath $statusFile
46     }
47
48     $checker = Join-Path $RepoRoot 'scripts\nightly_regression.py'
49     $manifest = Join-Path $RepoRoot 'output\human_lovo_manifest.json'
50
51     # Graceful no-op until the checker + corpus are present (pre-merge / corpus absent).
52     if (-not (Test-Path $checker)) {
53         "$date] nightly_regression.py not present at $checker ? skipping (awaiting merge)."
54     } | Tee-Object -FilePath $log
55     Write-Status 'SKIPPED' 0 'checker absent (awaiting merge)'
56     exit 0
57
58     if (-not (Test-Path $manifest)) {
59         "$date] golden manifest not present at $manifest ? skipping (corpus absent on this
60 machine)." | Tee-Object -FilePath $log
61     Write-Status 'SKIPPED' 0 'golden corpus absent'
62     exit 0
63
64     }
65
66     Set-Location $RepoRoot
67     if ($Pull) {
68         "$date] git pull --ff-only" | Tee-Object -FilePath $log -Append
69         & git -C $RepoRoot pull --ff-only 2>&1 | Tee-Object -FilePath $log -Append
70     }
71
72     $head = (& git -C $RepoRoot rev-parse --short HEAD 2>$null)
73     "$date] nightly rally-quality regression ? HEAD $head" | Tee-Object -FilePath $log
74 -Append
75
76     & $PythonExe $checker 2>&1 | Tee-Object -FilePath $log -Append
77     $code = $LASTEXITCODE
78
79     switch ($code) {
80         0 { Write-Status 'PASS' 0 "report: $reportDir\$date.md" }
81         1 { Write-Status 'REGRESSION' 1 "report: $reportDir\$date.md"
82         "$date] *** NIGHTLY REGRESSION DETECTED ? see $reportDir\$date.md ***" | Tee-
83 Object -FilePath $log -Append }
84         4 { Write-Status 'STALE' 4 "baseline refresh needed ? see $reportDir\$date.md" }
85         default { Write-Status 'ERROR' $code "checker error ? see $log" }
86     }
87
88     exit $code
89
90

```

7. user (2026-06-22T09:08:12.589Z)

```

1     <#
2     .SYNOPSIS
3         Register (or remove) the nightly rally-quality regression as a Windows Scheduled
4 Task.
5
6     .DESCRIPTION
7         Creates a daily user-level task that runs scripts\nightly_regression.ps1 against a

```

```

repo
7      checkout. The action is inline-guarded, so it's safe to register BEFORE this work is
merged
8      into the target checkout: it no-ops + logs "awaiting" until the wrapper exists
there, then
9      self-activates. Idempotent (-Force re-registers); reversible (-Unregister).
10
11      Registers under the current user (no admin / no -RunLevel Highest needed ?
consistent with the
12      repo's other scheduled tasks). Uses -StartWhenAvailable so a run missed while the
machine slept
13      is caught up on wake.
14
15      .EXAMPLE
16      # Register against the main checkout, nightly at 03:00:
17      pwsh -File scripts/register_nightly_task.ps1 -RepoRoot
'C:\Users\avidu\Projects\badminton-highlight-indexer'
18
19      .EXAMPLE
20      # Remove it:
21      pwsh -File scripts/register_nightly_task.ps1 -Unregister
22      #>
23      [CmdletBinding()]
24      param(
25          [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
26          [string]$TaskName = 'KhelSutraNightlyRegression',
27          [string]$Time = '03:00',
28          [switch]$Pull,
29          [switch]$Unregister
30      )
31
32      if ($Unregister) {
33          Unregister-ScheduledTask -TaskName $TaskName -Confirm:$false -ErrorAction
SilentlyContinue
34          Write-Host "Unregistered scheduled task '$TaskName'."
35          return
36      }
37
38      $wrapper = Join-Path $RepoRoot 'scripts\nightly_regression.ps1'
39      $ps = (Get-Command pwsh.exe -ErrorAction SilentlyContinue).Source
40      if (-not $ps) { $ps = 'powershell.exe' }
41
42      # Inline-guarded action: call the wrapper if present, else log "awaiting merge" and exit
0.
43      $marker = Join-Path $RepoRoot 'output\nightly_regression\LATEST_STATUS.txt'
44      $pullArg = if ($Pull) { ' -Pull' } else { '' }
45      $inner = @"
46      `Sw = '$wrapper'
47      if (Test-Path `Sw) { & `Sw -RepoRoot '$RepoRoot'$pullArg }
48      else {
49          New-Item -ItemType Directory -Force -Path (Split-Path -Parent '$marker') | Out-Null
50          "`$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss') status=SKIPPED exit=0 wrapper absent
(awaiting merge) at '$wrapper'" | Out-File -FilePath '$marker' -Encoding utf8
51      }
52      "@
53      $encoded = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($inner))
54
55      $action = New-ScheduledTaskAction -Execute $ps `
56          -Argument "-NoProfile -ExecutionPolicy Bypass -EncodedCommand $encoded"
57      $trigger = New-ScheduledTaskTrigger -Daily -At $Time
58      $settings = New-ScheduledTaskSettingsSet -StartWhenAvailable -DontStopOnIdleEnd `
59          -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries `
60          -ExecutionTimeLimit (New-TimeSpan -Minutes 30)
61      $principal = New-ScheduledTaskPrincipal -UserId "$env:USERDOMAIN\$env:USERNAME"

```

```

-LogonType Interactive -RunLevel Limited
62
63     Register-ScheduledTask -TaskName $TaskName -Action $action -Trigger $trigger `
64         -Settings $settings -Principal $principal -Force `
65         -Description "Nightly rally-quality regression tripwire: re-scores the golden corpus
(cached trajectories, CPU) vs eval_baselines/nightly_baseline.json. Writes
output/nightly_regression/<date>.md. Exit 1=regression, 4=stale baseline." | Out-Null
66
67     Write-Host "Registered scheduled task '$TaskName' ? daily at $Time, RepoRoot=$RepoRoot."
68     Write-Host "Wrapper: $wrapper"
69     Write-Host "Run now to test: Start-ScheduledTask -TaskName '$TaskName'"
70     Write-Host "Remove:          pwsh -File scripts/register_nightly_task.ps1 -Unregister"
71

```

8. assistant (2026-06-22T09:08:14.815Z)

Now let me examine the baseline file and understand the full structure:

9. user (2026-06-22T09:08:17.294Z)

```

1      {
2      "doc": "Frozen nightly rally-quality regression baseline. Regenerate with `python
scripts/nightly_regression.py --update-baseline` after an intended improvement or a corpus
change. Tied to the LOCAL golden corpus (not committed).",
3      "configs": {
4          "baseline_off": {
5              "aggregate": {
6                  "dend_abs_median": 41.77921047714383,
7                  "dstart_abs_median": 42.467243955066174,
8                  "f1@0.25": 0.39981006941991765,
9                  "f1@0.5": 0.2366145803119009,
10                 "merge_rate": 0.6380248744753217,
11                 "precision": 0.2577557018251538,
12                 "recall": 0.22719343607869802,
13                 "segment_count_ratio": 0.5824947574930804,
14                 "split_rate": 0.0,
15                 "tol_f1": 0.1382064040388172,
16                 "tol_precision": 0.14536686157033415,
17                 "tol_recall": 0.13445461315285676
18             },
19             "iou": 0.5,
20             "min_crossings": 0,
21             "n": 15,
22             "note": "Pre-R1-flip all-OFF windowing \u2014 the historical floor; pins the value
the milestone buys.",
23             "per_video": {
24                 "Badminton_BXH_2": {
25                     "f1@0.25": 0.0,
26                     "f1@0.5": 0.0,
27                     "merge_count": 3,
28                     "merge_rate": 1.0,
29                     "num_gt": 44,
30                     "num_pred": 4,
31                     "split_count": 0,
32                     "tol_f1": 0.0
33                 },
34                 "Boxhill_Doubles": {
35                     "f1@0.25": 0.10526315789473685,
36                     "f1@0.5": 0.03508771929824561,
37                     "merge_count": 7,
38                     "merge_rate": 0.9534883720930233,
39                     "num_gt": 43,
40                     "num_pred": 14,
41                     "split_count": 0,
42                     "tol_f1": 0.0

```



```
43     },
44     "GX010128": {
45         "f1@0.25": 0.14925373134328357,
46         "f1@0.5": 0.05970149253731343,
47         "merge_count": 12,
48         "merge_rate": 0.9615384615384616,
49         "num_gt": 52,
50         "num_pred": 15,
51         "split_count": 0,
52         "tol_f1": 0.0
53     },
54     "GX010137": {
55         "f1@0.25": 0.9253731343283583,
56         "f1@0.5": 0.5970149253731343,
57         "merge_count": 3,
58         "merge_rate": 0.17647058823529413,
59         "num_gt": 34,
60         "num_pred": 33,
61         "split_count": 0,
62         "tol_f1": 0.4477611940298507
63     },
64     "GX010141": {
65         "f1@0.25": 0.5416666666666666,
66         "f1@0.5": 0.3333333333333333,
67         "merge_count": 5,
68         "merge_rate": 0.6551724137931034,
69         "num_gt": 29,
70         "num_pred": 19,
71         "split_count": 0,
72         "tol_f1": 0.3333333333333333
73     },
74     "GX020094": {
75         "f1@0.25": 0.7333333333333334,
76         "f1@0.5": 0.4444444444444445,
77         "merge_count": 3,
78         "merge_rate": 0.15,
79         "num_gt": 40,
80         "num_pred": 50,
81         "split_count": 0,
82         "tol_f1": 0.2888888888888889
83     },
84     "GX030094": {
85         "f1@0.25": 0.5352112676056339,
86         "f1@0.5": 0.28169014084507044,
87         "merge_count": 8,
88         "merge_rate": 0.5609756097560976,
89         "num_gt": 41,
90         "num_pred": 30,
91         "split_count": 0,
92         "tol_f1": 0.14084507042253522
93     },
94     "adarsh_avi_singles": {
95         "f1@0.25": 0.8601036269430052,
96         "f1@0.5": 0.6321243523316062,
97         "merge_count": 5,
98         "merge_rate": 0.10416666666666667,
99         "num_gt": 96,
100        "num_pred": 97,
101        "split_count": 0,
102        "tol_f1": 0.38341968911917096
103    },
104    "gbaaddy": {
105        "f1@0.25": 0.17142857142857143,
106        "f1@0.5": 0.08571428571428572,
107        "merge_count": 9,
```

```
108         "merge_rate": 0.8541666666666666,
109         "num_gt": 48,
110         "num_pred": 22,
111         "split_count": 0,
112         "tol_f1": 0.0
113     },
114     "kushagra_singles": {
115         "f1@0.25": 0.0,
116         "f1@0.5": 0.0,
117         "merge_count": 2,
118         "merge_rate": 1.0,
119         "num_gt": 32,
120         "num_pred": 2,
121         "split_count": 0,
122         "tol_f1": 0.0
123     },
124     "targettest_doubles": {
125         "f1@0.25": 0.8118811881188118,
126         "f1@0.5": 0.5346534653465348,
127         "merge_count": 4,
128         "merge_rate": 0.16326530612244897,
129         "num_gt": 49,
130         "num_pred": 52,
131         "split_count": 0,
132         "tol_f1": 0.297029702970297
133     },
134     "mahadevpura_1": {
135         "f1@0.25": 0.0,
136         "f1@0.5": 0.0,
137         "merge_count": 1,
138         "merge_rate": 1.0,
139         "num_gt": 10,
140         "num_pred": 1,
141         "split_count": 0,
142         "tol_f1": 0.0
143     },
144     "mahadevpura_2": {
145         "f1@0.25": 0.0,
146         "f1@0.5": 0.0,
147         "merge_count": 3,
148         "merge_rate": 0.975,
149         "num_gt": 40,
150         "num_pred": 5,
151         "split_count": 0,
152         "tol_f1": 0.0
153     },
154     "mahadevpura_singles": {
155         "f1@0.25": 0.6181818181818182,
156         "f1@0.5": 0.3636363636363636,
157         "merge_count": 6,
158         "merge_rate": 0.5161290322580645,
159         "num_gt": 31,
160         "num_pred": 24,
161         "split_count": 0,
162         "tol_f1": 0.1818181818181818
163     },
164     "testlarge_short": {
165         "f1@0.25": 0.5454545454545454,
166         "f1@0.5": 0.1818181818181818,
167         "merge_count": 1,
168         "merge_rate": 0.5,
169         "num_gt": 6,
170         "num_pred": 5,
171         "split_count": 0,
172         "tol_f1": 0.0
```

```

173     }
174 },
175 "preset": {
176     "blob_factor": 4.0,
177     "blob_fallback": false,
178     "hard_cap": false,
179     "inactivity_end": 0.0,
180     "inactivity_min_density": 0.34,
181     "inactivity_rally_thresh": 0.08,
182     "max_window_duration": 0.0,
183     "merge_gap": 2.0,
184     "min_window_duration": 2.0,
185     "name": "baseline_off",
186     "velocity_thresh": 0.02
187 }
188 },
189 "default": {
190     "aggregate": {
191         "dend_abs_median": 2.9720372594817066,
192         "dstart_abs_median": 3.277048437326215,
193         "f1@0.25": 0.6296769405815345,
194         "f1@0.5": 0.4681709925781376,
195         "merge_rate": 0.13955677464298155,
196         "precision": 0.39945838409296397,
197         "recall": 0.5917335962330964,
198         "segment_count_ratio": 1.4784540509868136,
199         "split_rate": 0.11823489673413166,
200         "tol_f1": 0.3532006050209596,
201         "tol_precision": 0.30232209566271007,
202         "tol_recall": 0.44486191614359216
203     },
204     "iou": 0.5,
205     "min_crossings": 0,
206     "n": 15,
207     "note": "SERVED production windowing \u2014 now the R1 milestone (cap=6 + R4)
after the #204 flip.",
208     "per_video": {
209         "Badminton_BXH_2": {
210             "f1@0.25": 0.6721311475409836,
211             "f1@0.5": 0.4426229508196722,
212             "merge_count": 2,
213             "merge_rate": 0.09090909090909091,
214             "num_gt": 44,
215             "num_pred": 78,
216             "split_count": 4,
217             "tol_f1": 0.3278688524590163
218         },
219         "Boxhill_Doubles": {
220             "f1@0.25": 0.5538461538461538,
221             "f1@0.5": 0.4307692307692308,
222             "merge_count": 0,
223             "merge_rate": 0.0,
224             "num_gt": 43,
225             "num_pred": 87,
226             "split_count": 9,
227             "tol_f1": 0.26153846153846155
228         },
229         "GX010128": {
230             "f1@0.25": 0.7218045112781956,
231             "f1@0.5": 0.6015037593984963,
232             "merge_count": 0,
233             "merge_rate": 0.0,
234             "num_gt": 52,
235             "num_pred": 81,
236             "split_count": 2,

```

```
237     "tol_f1": 0.46616541353383456
238 },
239 "GX010137": {
240     "f1@0.25": 0.9041095890410958,
241     "f1@0.5": 0.8219178082191781,
242     "merge_count": 0,
243     "merge_rate": 0.0,
244     "num_gt": 34,
245     "num_pred": 39,
246     "split_count": 1,
247     "tol_f1": 0.6849315068493151
248 },
249 "GX010141": {
250     "f1@0.25": 0.7450980392156864,
251     "f1@0.5": 0.5098039215686274,
252     "merge_count": 6,
253     "merge_rate": 0.4482758620689655,
254     "num_gt": 29,
255     "num_pred": 22,
256     "split_count": 0,
257     "tol_f1": 0.4313725490196078
258 },
259 "GX020094": {
260     "f1@0.25": 0.6666666666666665,
261     "f1@0.5": 0.4242424242424242,
262     "merge_count": 0,
263     "merge_rate": 0.0,
264     "num_gt": 40,
265     "num_pred": 59,
266     "split_count": 7,
267     "tol_f1": 0.36363636363636365
268 },
269 "GX030094": {
270     "f1@0.25": 0.6451612903225806,
271     "f1@0.5": 0.4677419354838709,
272     "merge_count": 0,
273     "merge_rate": 0.0,
274     "num_gt": 41,
275     "num_pred": 83,
276     "split_count": 6,
277     "tol_f1": 0.41935483870967744
278 },
279 "adarsh_avi_singles": {
280     "f1@0.25": 0.7873303167420814,
281     "f1@0.5": 0.6787330316742081,
282     "merge_count": 0,
283     "merge_rate": 0.0,
284     "num_gt": 96,
285     "num_pred": 125,
286     "split_count": 13,
287     "tol_f1": 0.5520361990950227
288 },
289 "gbaaddy": {
290     "f1@0.25": 0.6388888888888889,
291     "f1@0.5": 0.4166666666666667,
292     "merge_count": 1,
293     "merge_rate": 0.041666666666666664,
294     "num_gt": 48,
295     "num_pred": 96,
296     "split_count": 8,
297     "tol_f1": 0.2777777777777778
298 },
299 "kushagra_singles": {
300     "f1@0.25": 0.6753246753246752,
301     "f1@0.5": 0.41558441558441556,
```

```

302         "merge_count": 3,
303         "merge_rate": 0.1875,
304         "num_gt": 32,
305         "num_pred": 45,
306         "split_count": 6,
307         "tol_f1": 0.07792207792207793
308     },
309     "targettest_doubles": {
310         "f1@0.25": 0.8,
311         "f1@0.5": 0.6666666666666666,
312         "merge_count": 0,
313         "merge_rate": 0.0,
314         "num_gt": 49,
315         "num_pred": 71,
316         "split_count": 5,
317         "tol_f1": 0.5833333333333334
318     },
319     "mahadevpura_1": {
320         "f1@0.25": 0.0,
321         "f1@0.5": 0.0,
322         "merge_count": 2,
323         "merge_rate": 0.8,
324         "num_gt": 10,
325         "num_pred": 2,
326         "split_count": 0,
327         "tol_f1": 0.0
328     },
329     "mahadevpura_2": {
330         "f1@0.25": 0.4878048780487805,
331         "f1@0.5": 0.24390243902439024,
332         "merge_count": 9,
333         "merge_rate": 0.525,
334         "num_gt": 40,
335         "num_pred": 42,
336         "split_count": 4,
337         "tol_f1": 0.12195121951219512
338     },
339     "mahadevpura_singles": {
340         "f1@0.25": 0.7469879518072289,
341         "f1@0.5": 0.6024096385542168,
342         "merge_count": 0,
343         "merge_rate": 0.0,
344         "num_gt": 31,
345         "num_pred": 52,
346         "split_count": 7,
347         "tol_f1": 0.5301204819277108
348     },
349     "testlarge_short": {
350         "f1@0.25": 0.4,
351         "f1@0.5": 0.3,
352         "merge_count": 0,
353         "merge_rate": 0.0,
354         "num_gt": 6,
355         "num_pred": 14,
356         "split_count": 1,
357         "tol_f1": 0.2
358     }
359 },
360 "preset": {
361     "blob_factor": 4.0,
362     "blob_fallback": false,
363     "hard_cap": false,
364     "inactivity_end": 1.5,
365     "inactivity_min_density": 0.3,
366     "inactivity_rally_thresh": 0.2,

```

```

367         "max_window_duration": 6.0,
368         "merge_gap": 2.0,
369         "min_window_duration": 2.0,
370         "name": "served",
371         "velocity_thresh": 0.02
372     }
373 },
374     "milestone": {
375         "aggregate": {
376             "dend_abs_median": 2.9720372594817066,
377             "dstart_abs_median": 3.277048437326215,
378             "f1@0.25": 0.6296769405815345,
379             "f1@0.5": 0.4681709925781376,
380             "merge_rate": 0.13955677464298155,
381             "precision": 0.39945838409296397,
382             "recall": 0.5917335962330964,
383             "segment_count_ratio": 1.4784540509868136,
384             "split_rate": 0.11823489673413166,
385             "tol_f1": 0.3532006050209596,
386             "tol_precision": 0.30232209566271007,
387             "tol_recall": 0.44486191614359216
388         },
389         "iou": 0.5,
390         "min_crossings": 0,
391         "n": 15,
392         "note": "R3 cap=6 s + R4 rally-END inactivity trim \u2014 the shipped milestone
(== default post-#204).",
393         "per_video": {
394             "Badminton_BXH_2": {
395                 "f1@0.25": 0.6721311475409836,
396                 "f1@0.5": 0.4426229508196722,
397                 "merge_count": 2,
398                 "merge_rate": 0.09090909090909091,
399                 "num_gt": 44,
400                 "num_pred": 78,
401                 "split_count": 4,
402                 "tol_f1": 0.3278688524590163
403             },
404             "Boxhill_Doubles": {
405                 "f1@0.25": 0.5538461538461538,
406                 "f1@0.5": 0.4307692307692308,
407                 "merge_count": 0,
408                 "merge_rate": 0.0,
409                 "num_gt": 43,
410                 "num_pred": 87,
411                 "split_count": 9,
412                 "tol_f1": 0.26153846153846155
413             },
414             "GX010128": {
415                 "f1@0.25": 0.7218045112781956,
416                 "f1@0.5": 0.6015037593984963,
417                 "merge_count": 0,
418                 "merge_rate": 0.0,
419                 "num_gt": 52,
420                 "num_pred": 81,
421                 "split_count": 2,
422                 "tol_f1": 0.46616541353383456
423             },
424             "GX010137": {
425                 "f1@0.25": 0.9041095890410958,
426                 "f1@0.5": 0.8219178082191781,
427                 "merge_count": 0,
428                 "merge_rate": 0.0,
429                 "num_gt": 34,
430                 "num_pred": 39,

```

```

431         "split_count": 1,
432         "tol_f1": 0.6849315068493151
433     },
434     "GX010141": {
435         "f1@0.25": 0.7450980392156864,
436         "f1@0.5": 0.5098039215686274,
437         "merge_count": 6,
438         "merge_rate": 0.4482758620689655,
439         "num_gt": 29,
440         "num_pred": 22,
441         "split_count": 0,
442         "tol_f1": 0.4313725490196078
443     },
444     "GX020094": {
445         "f1@0.25": 0.6666666666666665,
446         "f1@0.5": 0.4242424242424242,
447         "merge_count": 0,
448         "merge_rate": 0.0,
449         "num_gt": 40,
450         "num_pred": 59,
451         "split_count": 7,
452         "tol_f1": 0.36363636363636365
453     },
454     "GX030094": {
455         "f1@0.25": 0.6451612903225806,
456         "f1@0.5": 0.4677419354838709,
457         "merge_count": 0,
458         "merge_rate": 0.0,
459         "num_gt": 41,
460         "num_pred": 83,
461         "split_count": 6,
462         "tol_f1": 0.41935483870967744
463     },
464     "adarsh_avi_singles": {
465         "f1@0.25": 0.7873303167420814,
466         "f1@0.5": 0.6787330316742081,
467         "merge_count": 0,
468         "merge_rate": 0.0,
469         "num_gt": 96,
470         "num_pred": 125,
471         "split_count": 13,
472         "tol_f1": 0.5520361990950227
473     },
474     "gbaaddy": {
475         "f1@0.25": 0.6388888888888889,
476         "f1@0.5": 0.4166666666666667,
477         "merge_count": 1,
478         "merge_rate": 0.04166666666666664,
479         "num_gt": 48,
480         "num_pred": 96,
481         "split_count": 8,
482         "tol_f1": 0.2777777777777778
483     },
484     "kushagra_singles": {
485         "f1@0.25": 0.6753246753246752,
486         "f1@0.5": 0.41558441558441556,
487         "merge_count": 3,
488         "merge_rate": 0.1875,
489         "num_gt": 32,
490         "num_pred": 45,
491         "split_count": 6,
492         "tol_f1": 0.07792207792207793
493     },
494     "largetest_doubles": {
495         "f1@0.25": 0.8,

```

```
496         "f1@0.5": 0.6666666666666666,
497         "merge_count": 0,
498         "merge_rate": 0.0,
499         "num_gt": 49,
500         "num_pred": 71,
501         "split_count": 5,
502         "tol_f1": 0.5833333333333334
503     },
504     "mahadevpura_1": {
505         "f1@0.25": 0.0,
506         "f1@0.5": 0.0,
507         "merge_count": 2,
508         "merge_rate": 0.8,
509         "num_gt": 10,
510         "num_pred": 2,
511         "split_count": 0,
512         "tol_f1": 0.0
513     },
514     "mahadevpura_2": {
515         "f1@0.25": 0.4878048780487805,
516         "f1@0.5": 0.24390243902439024,
517         "merge_count": 9,
518         "merge_rate": 0.525,
519         "num_gt": 40,
520         "num_pred": 42,
521         "split_count": 4,
522         "tol_f1": 0.12195121951219512
523     },
524     "mahadevpura_singles": {
525         "f1@0.25": 0.7469879518072289,
526         "f1@0.5": 0.6024096385542168,
527         "merge_count": 0,
528         "merge_rate": 0.0,
529         "num_gt": 31,
530         "num_pred": 52,
531         "split_count": 7,
532         "tol_f1": 0.5301204819277108
533     },
534     "testlarge_short": {
535         "f1@0.25": 0.4,
536         "f1@0.5": 0.3,
537         "merge_count": 0,
538         "merge_rate": 0.0,
539         "num_gt": 6,
540         "num_pred": 14,
541         "split_count": 1,
542         "tol_f1": 0.2
543     }
544 },
545 "preset": {
546     "blob_factor": 4.0,
547     "blob_fallback": false,
548     "hard_cap": false,
549     "inactivity_end": 1.5,
550     "inactivity_min_density": 0.3,
551     "inactivity_rally_thresh": 0.2,
552     "max_window_duration": 6.0,
553     "merge_gap": 2.0,
554     "min_window_duration": 2.0,
555     "name": "milestone",
556     "velocity_thresh": 0.02
557 }
558 }
559 },
560 "generated_at": "2026-06-19",
```



```
561     "git_commit": "a240221",
562     "iou": 0.5,
563     "manifest": "output/human_lovo_manifest.json",
564     "tolerances": {
565         "metric_tol": 0.005,
566         "over_seg_tol": 0,
567         "rate_tol": 0.01
568     }
569 }
570
```

10. user (2026-06-22T09:08:17.757Z)

(Bash completed with no output)

11. assistant (2026-06-22T09:08:21.175Z)

Let me look at the segmenter/pipeline interfaces to understand where reel counts would come from:

12. user (2026-06-22T09:08:23.182Z)

Found 53 files
tools\cleanup_caches.py
tests\test_config.py
tests\test_cleanup_caches.py
tests\test_api_endpoints.py
backend\main.py
tests\test_app_home.py
backend\utils\app_home.py
backend\config\models.py
backend\billing.py
tests\test_compute_backend.py
backend\worker__main__.py
tests\test_output_base.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
tests\test_worker.py
backend\utils\workspace.py
tests\test_workspace.py
tests\test_generate_highlights.py
artifacts\so.py
backend\pipeline\segmenters\fusion_hybrid.py
scripts\run_process_and_stitch.py
backend\pipeline\stitcher\compiler.py
artifacts\landing\make_og.py
backend\eval\rally_seg_eval.py
tests\test_rally_seg_eval.py
backend\providers\gemini.py
tools\generate_highlights.py
tests\test_rally_seg_eval_report.py
tests\test_experiment.py
backend\eval\golden_fixtures.py
backend\eval\experiment.py
backend\eval\ablation.py
tests\test_served_gate_a_regression.py
tests\test_upload_download.py
backend\eval\ablation_pdf.py
tests\test_nightly_regression.py
scripts\nightly_regression.py
docs\archives\quality-iterations\2026-06-18-lovo-resweep-n13\lovo_resweep.py
backend\eval\tolerance_metrics.py
tests\test_segmentation_metrics.py

```
backend\eval\segmentation_metrics.py
tests\test_compiler.py
backend\eval\served_gate_a.py
tests\test_experiment_honest.py
backend\tools\export.py
tests\test_watermark.py
tests\test_path_safety.py
backend\reporting\harvest.py
tests\test_reporting_harvest.py
run_all_tests.py
backend\tools\rally_slicer.py
backend\sports\badminton.py
```

13. user (2026-06-22T09:08:23.348Z)

EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\nightly-e2e\backend\pipeline\segmenters'

14. user (2026-06-22T09:08:26.230Z)

```
total 121
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 .
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 ..
-rw-r--r-- 1 avidu 197609   757 Jun 22 14:36 __init__.py
-rw-r--r-- 1 avidu 197609   546 Jun 22 14:36 _keyhint.py
-rw-r--r-- 1 avidu 197609  2420 Jun 22 14:36 base.py
-rw-r--r-- 1 avidu 197609 11618 Jun 22 14:36 fusion_hybrid.py
-rw-r--r-- 1 avidu 197609 26428 Jun 22 14:36 gemini.py
-rw-r--r-- 1 avidu 197609 11019 Jun 22 14:36 motion.py
-rw-r--r-- 1 avidu 197609   1748 Jun 22 14:36 tracknet_hybrid.py
-rw-r--r-- 1 avidu 197609 20341 Jun 22 14:36 trajectory_hybrid.py
-rw-r--r-- 1 avidu 197609   2329 Jun 22 14:36 wasb_hybrid.py
-rw-r--r-- 1 avidu 197609 21340 Jun 22 14:36 yolo_hybrid.py
```

15. user (2026-06-22T09:08:27.468Z)

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/audio/hit_detector.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/audio/__init__.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/audio_features.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/base.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/court_gate.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/device.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/fusion_features.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/native_wasb_runner.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/person_detector.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/rally_gate.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/stub_runner.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/tracknet_runner.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/wasb_infer.py
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/wasb_runner.py
```

```

C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/detectors/__init__.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/personalization/serving.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/personalization/__init__.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/base.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/fusion_hybrid.py
C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e/backend/pipeline/segmenters/gemini.py

```

16. user (2026-06-22T09:08:30.018Z)

```

1      from abc import ABC, abstractmethod
2      from typing import List, Dict, Any, Type, Optional
3      from backend.infrastructure.database import Database
4      from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6      class BasePlaySegmenter(ABC):
7          def __init__(self, db: Database, config: Dict[str, Any]):
8              self.db = db
9              self.config = config
10
11          @property
12          @abstractmethod
13          def name(self) -> str:
14              """Unique identifier for this segmenter."""
15              pass
16
17          @property
18          @abstractmethod
19          def description(self) -> str:
20              """Human-readable explanation of this segmenter."""
21              pass
22
23          @abstractmethod
24          def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25              """
26              Processes a video to detect play segments, populates SQLite database,
27              and returns a Result.
28
29              Use isinstance(result, Failure) (or result.is_success()) to distinguish
30              "no segments found" from "something went wrong" (missing key, healthcheck
31              failure, etc.). This is the start of the standardized error/result contract.
32              """
33              pass
34
35      class SegmenterRegistry:
36          def __init__(self):
37              self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38
39          def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40              """Registers a new segmenter class under its name property."""
41              try:
42                  temp_inst = segmenter_cls(None, {})
43                  name = temp_inst.name
44              except Exception as e:
45                  raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46              self._segmenters[name] = segmenter_cls
47

```

```

48     def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49         return self._segmenters.get(name)
50
51     def list_available(self) -> Dict[str, str]:
52         """Returns a dict of registered segmenter names and their descriptions."""
53         res = {}
54         for name, cls in self._segmenters.items():
55             try:
56                 temp_inst = cls(None, {})
57                 desc = temp_inst.description
58             except Exception:
59                 desc = "No description available."
60             res[name] = desc
61         return res
62
63     segmenter_registry = SegmenterRegistry()
64

```

17. user (2026-06-22T09:08:31.147Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/scripts/run_process_and_stitch.py

18. user (2026-06-22T09:08:34.318Z)

```

1     import os
2     import json
3     import logging
4     from dotenv import load_dotenv
5
6     load_dotenv()
7
8     # Central root-logger setup ? all backend modules use logging.getLogger(__name__).
9     # Quiet third-party HTTP chatter by default; re-honoured against the config knob
10    # (logging.verbose_http) once the config is loaded in main().
11    from backend.utils.logging_setup import setup_logging
12    setup_logging()
13
14    from backend.infrastructure.database import Database
15    from backend.config import load_config
16    from backend.utils.hashing import compute_video_id
17    from backend.pipeline.segmenters import segmenter_registry
18    from backend.pipeline.stitcher.compiler import VideoCompiler
19    from backend.results import unwrap_or_failure
20    from backend.tools.export import format_rally_summary
21
22    logger = logging.getLogger(__name__)
23
24    def main():
25        import argparse
26        parser = argparse.ArgumentParser(
27            description="Dev runner: process a video -> index rallies -> stitch a highlight
28            reel.")
29        parser.add_argument("--video-path", required=True, help="Path to the input video
30            file.")
31        parser.add_argument("--output-dir", default=None,
32            help="Where to write the stitched reel (default: config
33            output.output_dir).")
34        parser.add_argument("--output-filename", default="highlights.mp4", help="Stitched
35            reel filename.")
36        args = parser.parse_args()
37        video_path = os.path.abspath(args.video_path)
38        output_filename = args.output_filename
39
40        print("Loading config...")

```

```

37     cfg = load_config() # typed AppConfig ? single source of defaults
38     # Re-apply now that the verbose_http knob is known (default keeps HTTP chatter
quiet).
39     setup_logging(verbose_http=cfg.logging.verbose_http)
40     output_dir = args.output_dir or cfg.output.output_dir # no hardcoded OS path
41
42     print("Verifying files exist...")
43     if not os.path.exists(video_path):
44         print(f"Error: Video file not found at {video_path}")
45         return
46
47     os.makedirs(cfg.output.output_dir, exist_ok=True)
48     db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
49     db = Database(db_path)
50
51     # Calculate MD5 of the video to uniquely identify it
52     import time
53     print("Calculating video MD5 (this might take a few seconds on large files)...",
flush=True)
54     t0 = time.time()
55     video_id = compute_video_id(video_path)
56     print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
57
58     segments = []
59     cached_video = db.get_video(video_id)
60     cache_hit = False
61
62     if cached_video and cached_video.get("status") == "processed":
63         segments = db.query_play_segments(video_id, {})
64         if len(segments) > 0:
65             print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.", flush=True)
66             choice = input("Would you like to (1) Run stitching on the cached segments,
or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
67             if choice == '1':
68                 print("[CACHE] Using cached play segments. Skipping segmenter API
call.", flush=True)
69                 cache_hit = True
70             else:
71                 print("[CACHE] Ignoring cache...", flush=True)
72         else:
73             print("[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
74
75     if not cache_hit:
76         print("[CACHE] Initializing clean indexing...", flush=True)
77         db.clear_video_data(video_id)
78
79     print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
80     db.add_video(video_id, video_path, "processed", [])
81
82     print("Running segmenter & indexing...", flush=True)
83     seg_name = cfg.indexing.default_segmenter
84     segmenter_cls = segmenter_registry.get(seg_name)
85     if not segmenter_cls:
86         print(f"[FAIL] Segmenter '{seg_name}' not found.")
87         return
88     print(f"Using segmenter: {seg_name}", flush=True)
89     segmenter = segmenter_cls(db, cfg)
90     # process_video returns a Result (Success|Failure) for some segmenters (e.g.
`motion`);
91     # unwrap before len()/truthiness so this doesn't crash with TypeError on a
Success
92     # dataclass (and so the empty-guard below actually sees the list) ? mirrors

```

```

backend/main.py.
93         segments, failure = unwrap_or_failure(segmenter.process_video(video_id,
video_path))
94         if failure is not None:
95             print(f"[FAIL] Segmentation failed: {failure.message}")
96             return
97         print(f"Successfully indexed {len(segments)} play segments.", flush=True)
98
99     if not segments:
100         print("No play segments detected. Cannot compile highlights.")

```

19. user (2026-06-22T09:08:35.274Z)

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/stitcher
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/backend/pipeline/stitcher/compiler.py

20. user (2026-06-22T09:08:37.811Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     from backend.utils.ffmpeg import ffmpeg_bin, ffprobe_bin # P2a: single ffmpeg/ffprobe
resolver
9
10    logger = logging.getLogger(__name__)
11
12    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
13    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
14    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
15    # backend/pipeline/stitcher/, so parents[2] == backend/.
16    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
17
18    class VideoCompiler:
19        def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
20                    watermark: Optional[Any] = None):
21            self.output_dir = output_dir
22            self.end_padding = end_padding
23            self.begin_padding = begin_padding
24            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
25            # normalized to a dict (or None) so this module stays config-package-agnostic.
26            self.watermark = self._normalize_watermark(watermark)
27            os.makedirs(output_dir, exist_ok=True)
28
29        @staticmethod
30        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
31            if watermark is None:
32                return None
33            if hasattr(watermark, "model_dump"):
34                return watermark.model_dump()
35            if isinstance(watermark, dict):
36                return dict(watermark)
37            return None
38
39        @staticmethod

```

```

40 def _ff_quote(path: str) -> str:
41     """Make a filesystem path safe to embed inside single quotes in an ffmpeg
42     filtergraph operand (drawtext fontfile/textfile, movie= source): forward
43     slashes (Windows-friendly), escaped single quotes, AND escaped colons.
44
45     The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
46     option parser treats ':' as the option separator, so a Windows drive-letter
47     path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
48     the encode fails with "Invalid argument". This silently disabled the
49     on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
50     return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
51
52 @staticmethod
53 def _parse_time(val: Union[int, float, str]) -> float:
54     """Normalizes a timestamp value to float seconds.
55     Handles: float/int (pass-through), plain numeric strings ("12.4"),
56     and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
57     This mirrors the parse_time logic used in gemini.py to ensure
58     the stitcher can always consume whatever format the indexer produced."""
59     if isinstance(val, (int, float)):
60         return float(val)
61     if isinstance(val, str):
62         val = val.strip()
63         if ":" in val:
64             # Handle MM:SS, HH:MM:SS, etc.
65             parts = val.split(":")
66             parts.reverse()
67             total = 0.0
68             for i, p in enumerate(parts):
69                 try:
70                     total += float(p) * (60 ** i)
71                 except ValueError:
72                     pass
73             return total
74         try:
75             return float(val)
76         except ValueError:
77             logger.warning(f"Could not parse timestamp value '{val}' as a number.
78             Defaulting to 0.0.")
79             return 0.0
80             logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
81             '{val}'. Defaulting to 0.0.")
82             return 0.0
83
84 @staticmethod
85 def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
86 float]]:
87     """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
88
89     A highlight reel must never replay the same footage. The same rally can land in
90     the segment list more than once ? e.g. two detector runs accumulated for one
91     video (add_video is an UPSERT, so re-running without a prune leaves both sets),
92     or
93     chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
94     rally
95     twice. Merging any range that overlaps or abuts the previous one collapses true
96     duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
97     Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
98     """
99     parsed = []
100     for raw_s, raw_e in segments:
101         s = VideoCompiler._parse_time(raw_s)
102         e = VideoCompiler._parse_time(raw_e)
103         if e > s:
104             parsed.append((s, e))

```

```

100         parsed.sort()
101         merged: List[Tuple[float, float]] = []
102         for s, e in parsed:
103             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
104                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
105             else:
106                 merged.append((s, e))
107         return merged
108
109     def _get_video_duration(self, video_path: str) -> float:
110         """Probes the video file to get its total duration in seconds.
111         Returns 0.0 if it cannot be determined."""
112         try:
113             result = subprocess.run(
114                 [
115                     ffprobe_bin(), "-v", "error",
116                     "-show_entries", "format=duration",
117                     "-of", "default=noprint_wrappers=1:nokey=1",
118                     video_path
119                 ],
120                 capture_output=True, text=True, check=True
121             )
122             return float(result.stdout.strip())
123         except Exception as e:
124             logger.warning(f"Could not determine video duration via ffprobe: {e}")
125             return 0.0
126
127     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
128         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
129         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
130         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
131         because the mark is already baked into every clip identically."""
132         wm = self.watermark
133         if not wm or not wm.get("enabled"):
134             return None
135
136         text = (wm.get("text") or "").strip()
137         logo_path = (wm.get("logo_path") or "").strip()
138         if logo_path and not os.path.exists(logo_path):
139             logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
140 overlay.")
141             logo_path = ""
142         if not text and not logo_path:
143             return None
144
145         margin = int(wm.get("margin", 24))
146         opacity = float(wm.get("opacity", 0.85))
147         position = str(wm.get("position", "bottom-right"))
148         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
149         w/h (logo).
150         dt_xy = {
151             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
152             "bottom-left": f"x={margin}:y=h-th-{margin}",

```

21. assistant (2026-06-22T09:08:39.959Z)

Let me check what output the compiler produces:

22. user (2026-06-22T09:08:42.328Z)

```

305:     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float, float]],
306:                             output_filename: str) -> str:
328:         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
394:         output_path
401:         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:

```



```
404:             f"but output file is missing or empty: {output_path}"
407:             output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
408:             logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
409:             return output_path
```

23. assistant (2026-06-22T09:08:44.349Z)

Let me look at the output directory structure and manifest:

24. user (2026-06-22T09:08:45.450Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

25. user (2026-06-22T09:08:46.399Z)

ls: cannot access 'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e/output/': No such file or directory

26. user (2026-06-22T09:08:50.029Z)

(Bash completed with no output)

27. assistant (2026-06-22T09:08:52.099Z)

Let me check the manifest location in the script:

28. user (2026-06-22T09:08:53.118Z)

```
./docs/archives/checkpoints/2026-06-eval-baseline
./eval_baselines
./eval_baselines/fixtures/manifest.json
./eval_baselines/nightly_baseline.json
```

29. user (2026-06-22T09:08:55.700Z)

```
1      [
2      {
3          "fps": 30.0,
4          "frame_width": 1280.0,
5          "kind": "synthetic",
6          "paths": {
7              "dir": "clean_single_rally",
8              "expected": "clean_single_rally/expected.json",
9              "labels": "clean_single_rally/labels.csv",
10             "provenance": "clean_single_rally/provenance.json",
11             "trajectory": "clean_single_rally/trajectory.csv"
12         },
13         "schema_version": 1,
14         "slug": "clean_single_rally",
15         "windowing": {
16             "max_win": 0.0,
17             "mg": 2.0,
18             "mwd": 2.0,
19             "vt": 0.02
20         }
21     },
22     {
23         "fps": 30.0,
24         "frame_width": 1280.0,
25         "kind": "synthetic",
26         "paths": {
27             "dir": "multi_rally_with_deadtime",
```

```

28         "expected": "multi_rally_with_deadtime/expected.json",
29         "labels": "multi_rally_with_deadtime/labels.csv",
30         "provenance": "multi_rally_with_deadtime/provenance.json",
31         "trajectory": "multi_rally_with_deadtime/trajectory.csv"
32     },
33     "schema_version": 1,
34     "slug": "multi_rally_with_deadtime",
35     "windowing": {
36         "max_win": 0.0,
37         "mg": 2.0,
38         "mwd": 2.0,
39         "vt": 0.02
40     }
41 },
42 {
43     "fps": 30.0,
44     "frame_width": 1280.0,
45     "kind": "synthetic",
46     "paths": {
47         "dir": "occlusion_break",
48         "expected": "occlusion_break/expected.json",
49         "labels": "occlusion_break/labels.csv",
50         "provenance": "occlusion_break/provenance.json",

```

30. assistant (2026-06-22T09:08:58.968Z)

Now let me understand the end-to-end pipeline and config structure:

31. user (2026-06-22T09:09:00.444Z)

```

backend/config/models.py: default_highlights_name: str = "highlights.mp4"
backend/config/models.py: # Directory where the DB, highlight reels, and processed clips are
backend/config/models.py: written.
backend/config/models.py: """Branding overlay burned into each highlight clip during the per-
clip re-encode
backend/config/models.py: (backend/pipeline/stitcher/compiler.py). **On by default** ? set
`enabled: false`
backend/config/models.py: (under `stitching.watermark` in config.json) to turn it off. The
text is fully
backend/config/models.py: # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally
pairs whose duration
backend/config/models.py: # (end ? start) is below this from the highlight reel, BEFORE
compiling. A cleaner
backend/config/models.py: # reel-selection knob than `min_shot_count`: shot_count is
unlabeled / "Unknown" on the
backend/config/models.py: # background-court fragments), so this keeps the reel to the eye-
catching long rallies.
backend/config/models.py: # OFF by default ? the detector output is unchanged, only which
rallies reach the reel.
backend/config/models.py: stitching: StitchingConfig = Field(default_factory=StitchingConfig)

```

32. user (2026-06-22T09:09:03.919Z)

```

1     """Pydantic models for the application configuration.
2
3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     The models support graceful migration from old config.json files via extra="allow".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional

```

```

12
13 from pydantic import BaseModel, Field, field_validator, model_validator
14
15 Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18 class ValidationRelevanceConfig(BaseModel):
19     court_green_lower: list[int] = Field(default=[35, 40, 40])
20     court_green_upper: list[int] = Field(default=[85, 255, 255])
21     court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24 class ValidationConfig(BaseModel):
25     min_resolution_width: int = 1280
26     min_resolution_height: int = 720
27     min_fps: float = 29.9
28     min_blur_threshold: float = 20.0
29     max_scene_cuts_per_minute: float = 0.5
30     relevance: ValidationRelevanceConfig =
31     Field(default_factory=ValidationRelevanceConfig)
32
33 class TrackNetConfig(BaseModel):
34     wsl_distro: str = "Ubuntu"
35     repo_dir: str = "~/models/TrackNetV4"
36     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37     conda_env: str = "TrackNetV4"
38     weights_path: str = ""
39     queue_length: int = 5
40     stage_in_wsl: bool = True
41     wsl_stage_dir: str = "~/clips"
42     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43     velocity_threshold: float = 0.02
44     # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45     # decoded frame cache) are deleted automatically ? they are pure intermediates,
46     # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47     # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48     keep_frames: bool = False
49     # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50     # 0 = disabled.
51     min_free_gb: float = 0.0
52
53
54 class WasbIndexConfig(BaseModel):
55     """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58     these knobs actually take effect ? previously this whole block was silently dropped
59     because nested config sections defaulted to extra='ignore'.
60     """
61     wsl_distro: str = "Ubuntu"
62     repo_dir: str = "~/models/WASB-SBDT"
63     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64     conda_env: str = "wasb"
65     weights_path: str = "~/models/WASB-
66     SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
67     sport: str = "badminton"
68     wsl_stage_dir: str = "~/clips"
69     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
70     velocity_threshold: float = 0.02
71     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
72     ---
73     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
74     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
75     CPU fallback

```

```

73         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
74         device: str = "cuda"          # auto | cuda | mps | cpu
75         python_bin: str = "python"    # the `wasb` conda env python (invoked by path, no
`conda activate`)
76         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77         # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
78         # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
79         keep_frames: bool = False
80         # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
81         min_free_gb: float = 0.0
82         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
83         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87         # Tri-state:
88         #     None          = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89         #                     behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90         #     True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91         stream_video: Optional[bool] = None
92
93
94     class FusionConfig(BaseModel):
95         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97         Every default reproduces control behaviour: the fusion segmenter persists the
98         shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99         true ? the person-cue features, but it does NOT gate the served handoff unless a
100        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102        supplied ? off-court persons (spectators / adjacent courts) are excluded.
103        """
104        # Which trajectory substrate seeds the windows (shuttle stays primary).
105        substrate: Literal["wasb", "tracknet"] = "wasb"
106        # Windowing velocity threshold for the fusion family (the engine reads
107        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109        velocity_threshold: float = 0.02
110        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111        # enabled ? so the default path never imports torch / touches the GPU.
112        cue_flags: dict[str, bool] = Field(default_factory=dict)
113        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115        min_crossings: int = Field(default=0, ge=0)
116        # Served Gate B: when the person cue is on, drop windows with median in-court
players
117        # < this. 0 = OFF.
118        min_players: int = Field(default=0, ge=0)

```

```

119     # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120     court_polygon: Optional[list[list[float]]] = None
121     # Net line for the person two-sided-motion split (person features only ? the shuttle
122     # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123     net_axis: Literal["x", "y"] = "y"
124     net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
135         pipeline/detectors/{stub,native_wasb}_runner.py.
136         detector_impl: str = "auto"
137         motion_threshold: float = 0.08
138         min_segment_duration: float = 3.0
139         dead_time_merge_gap: float = 2.0
140         chunk_duration: float = 90.0
141         chunk_overlap: float = 10.0
142         detector_model: str = "fasterrcnn_resnet50_fpn"
143         detector_score_threshold: float = 0.5
144         yolo_velocity_threshold: float = 0.15
145         yolo_frame_skip: int = 3
146         yolo_tracker: str = "bytetrack.yaml"
147         yolo_prefetch_workers: int = 2
148         min_action_window_duration: float = 2.0
149         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
150         # = a window
151         # longer than this many seconds is recursively split at its largest internal
152         # inactivity gap (?
153         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
154         # swallow
155         # several rallies + the dead time between them. Never merges, only cuts (recall
156         # can't drop).
157         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
158         # Measured n=13
159         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
160         # p=0.0005; cap=6
161         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
162         # 0.651?0.161). ***
163         max_window_duration: float = 6.0
164         window_min_split_gap: float = 0.5 # min internal gap (s) that qualifies as a split
165         point
166         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
167         # inactivity gap
168         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
169         # is
170         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
171         # bound.
172         # Only cuts (recall can't drop). OFF by default until measured/promoted.
173         window_hard_cap: bool = False
174         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
175         # slowly
176         # (picked up / walked) above velocity_thresh, so the window over-extends its END
177         # (the measured
178         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
179         # trailing
180         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
181         # cuts.
182         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
183         # significant

```

```

167         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
168         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169         window_inactivity_end: float = 1.5
170         inactivity_rally_thresh: float = 0.20
171         inactivity_min_density: float = 0.30
172         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180         window_blob_fallback: bool = False
181         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185         window_blob_factor: float = 4.0
186         log_yolo_candidates: bool = True
187         store_yolo_candidates: bool = True
188         ai_handoff_padding: float = 2.0
189         ai_max_workers: int = 5
190         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191         # chunks of a high-bitrate (4K) source blow past the provider upload cap
192         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195         # Matches gemini_refine's --clip-scale-width default.
196         ai_chunk_scale_width: int = 1280
197         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
200         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202         skip_ai_handoff: bool = False
203         # Optional debug knob: trim every video to the first N seconds before processing.
204         debug_duration: Optional[float] = None
205
206         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208         fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210         @field_validator("default_segmenter")
211         @classmethod
212         def segmenter_must_be_registered(cls, v: str) -> str:
213             # Registry check happens at runtime in main/segmenter selection.
214             # Here we just allow any string for forward compatibility.

```

```

215         return v
216
217     @model_validator(mode="after")
218     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
220         # step makes `while t < duration: t += step` loop forever, growing memory before
any
221         # GPU work. Reject the bad config at load time instead.
222         if self.chunk_overlap >= self.chunk_duration:
223             raise ValueError(
224                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
225                 f"({self.chunk_duration}); otherwise chunking never advances."
226             )
227         return self
228
229
230     class OutputConfig(BaseModel):
231         default_highlights_name: str = "highlights.mp4"
232         db_name: str = "sports_indexer.db"
233         # Directory where the DB, highlight reels, and processed clips are written.
234         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
235         output_dir: str = "output"
236
237
238     class WatermarkConfig(BaseModel):
239         """Branding overlay burned into each highlight clip during the per-clip re-encode
240         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
241         (under `stitching.watermark` in config.json) to turn it off. The text is fully
242         configurable. The concat stage stays stream-copy (-c copy)."""
243         enabled: bool = True
244         text: str = "Generated by Khelsutra.guru"
245         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
246         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family
below
247         font: str = "Sans" # fontconfig family used when font_path is empty
248         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5) # text height as a
fraction of frame height
249         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
250         box: bool = True # faint backing box behind the text for legibility

```

33. assistant (2026-06-22T09:09:06.161Z)

Let me look at the main entry point to understand how the full pipeline works:

34. user (2026-06-22T09:09:07.865Z)

```

279:     # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
381: def _reels_dir() -> str:
388: @app.get("/api/reels")
389: def list_reels():
390:     """List compiled highlight reels present in the output dir, newest first, with a
download URL.
392:     NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ? the
engine
393:     does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
394:     this is a global list. (Per-video reel tracking would need a DB record + compile change
? a
396:     base = _reels_dir()
397:     reels = []
401:     return {"reels": []}
409:     reels.append({

```

```

413:         "download_url": f"/api/reels/{quote(e.name)}",
415:     reels.sort(key=lambda r: r["modified_at"], reverse=True)
416:     return {"reels": reels}
419:@app.get("/api/reels/{filename}")
420:def download_reel(filename: str):
421:     """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
427:     base = _reels_dir()
434:         raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
513:         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
560:def process_video(req: ProcessRequest, request: Request):
672:     """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
706:def compile_highlights(req: CompileRequest):
752:     # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the
user's language
765:         out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
774:def run_cli_diagnose_clip(video_path: str, timestamp: float):
864:     parser.add_argument("--output-dir", default="output", help="Directory where processed
clips/highlights are saved")
866:     parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video
clip to run detailed diagnostics on")
919:         # clip carries the same configured stitching paddings + watermark as a real

```

35. user (2026-06-22T09:09:10.634Z)

```

706     def compile_highlights(req: CompileRequest):
707         video = db_instance.get_video(req.video_id)
708         if not video:
709             raise HTTPException(status_code=404, detail="Indexed video record not found.")
710
711         # Retrieve matching play segments from db
712         all_segments = db_instance.query_play_segments(req.video_id, {})
713         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
714
715         if not matched_segments:
716             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
717
718         # Reject path traversal / absolute output names (os.path.join would otherwise let an
719         # absolute or '..'-laden output_filename write anywhere on disk).
720         try:
721             out_name = safe_output_filename(req.output_filename)
722         except ValueError as e:
723             raise HTTPException(status_code=400, detail=str(e)) from e
724
725         # The configured shot-count floor (stitching.min_shot_count) applies on the API
726         # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
727         # metadata are exempt: the floor filters known counts only, so explicitly
728         # requested manual/legacy segments can't silently vanish under the default floor.
729         stitching = getattr(global_config, "stitching", None) or StitchingConfig()
730         kept = []
731         for s in matched_segments:
732             meta = s.get("metadata") or {}
733             sc = meta.get("shot_count") if isinstance(meta, dict) else None
734             try:
735                 if sc is not None and int(sc) < stitching.min_shot_count:
736                     continue
737             except (TypeError, ValueError):
738                 pass # unparseable count -> treat as unknown, keep
739             kept.append(s)
740         if len(kept) < len(matched_segments):
741             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped ")

```



```

742             f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
743         if not kept:
744             raise HTTPException(status_code=400,
745                                 detail=f"All {len(matched_segments)} matching segments fall
below "
746                                     f"stitching.min_shot_count={stitching.min_shot_count}.")
747
748         # Extract start/end time pairs
749         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
750
751         # Run compiler with the configured stitching paddings + optional branding watermark,
localized to
752         # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the
user's language
753         # with a script-appropriate font. Unknown/absent locale keeps the configured default
unchanged.
754         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
755         localized_watermark = localize_watermark(stitching.watermark, req.locale)
756         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
757                                 end_padding=stitching.end_padding,
watermark=localized_watermark)
758         try:
759             # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
760             # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
761             # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
762             # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
763             # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
764             local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
765             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
766             # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
767             # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
$4.3).
768             download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
769             return {"status": "success", "filepath": out_path, "download_url": download_url}
770         except Exception as e:
771             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
772
773     # --- CLI Debug Utility & Orchestration ---
774     def run_cli_diagnose_clip(video_path: str, timestamp: float):
775         """CLI debugging utility to examine segment properties around a timestamp."""
776         print("=" * 60)
777         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
778         print("=" * 60)
779
780         cfg = load_config() # legacy wrapper still works, returns dict for now
781
782         # 1. Validation check diagnostics
783         print("[DIAGNOSTIC] Running validation framework...")
784         results = registry.run_all(video_path, cfg)
785         for r in results:

```

36. assistant (2026-06-22T09:09:12.632Z)

Now let me check the test to understand the structure better:

37. user (2026-06-22T09:09:14.168Z)

(Bash completed with no output)

38. user (2026-06-22T09:09:16.634Z)

```
1      """Tests for scripts/nightly_regression.py ? the pure summary/comparison/report core.
2
3      Fully offline: no real golden corpus, no trajectory I/O, no GPU/numpy. The comparison
core
4      operates on plain summary dicts, so a regression / over-seg-guard / corpus-change can be
5      exercised deterministically. The real-corpus scoring path (`run_corpus`) is a thin
wrapper
6      over ``rally_seg_eval.evaluate`` (tested in test_rally_seg_eval.py) and is exercised e2e
on the
7      owner's machine, not here.
8      """
9      import importlib.util
10     import os
11     import sys
12
13     import pytest
14
15     # scripts/ isn't a package ? load the module by path. Register in sys.modules BEFORE
exec so
16     # dataclasses can resolve the `from __future__ import annotations` string annotations.
17     _SCRIPT = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
18                             "scripts", "nightly_regression.py")
19     _spec = importlib.util.spec_from_file_location("nightly_regression", _SCRIPT)
20     nr = importlib.util.module_from_spec(_spec)
21     sys.modules["nightly_regression"] = nr
22     _spec.loader.exec_module(nr)
23
24
25     # ----- #
26     # Builders
27     # ----- #
28     _PRESET = {"name": "x", "velocity_thresh": 0.02, "merge_gap": 2.0,
"min_window_duration": 2.0,
29                 "max_window_duration": 0.0, "hard_cap": False, "inactivity_end": 0.0,
30                 "inactivity_rally_thresh": 0.08, "inactivity_min_density": 0.34}
31
32
33     def _summary(aggregate, per_video, preset=None, min_crossings=0, n=None):
34         return {
35             "n": n if n is not None else len(per_video),
36             "preset": preset or dict(_PRESET),
37             "min_crossings": min_crossings,
38             "iou": 0.5,
39             "aggregate": dict(aggregate),
40             "per_video": {k: dict(v) for k, v in per_video.items()},
41         }
42
43
44     def _pv(tol_f1=0.3, split_count=2, merge_count=0, **extra):
45         d = {"tol_f1": tol_f1, "f1@0.5": 0.4, "f1@0.25": 0.5, "merge_rate": 0.1,
46             "split_count": split_count, "merge_count": merge_count, "num_pred": 10,
"num_gt": 10}
47         d.update(extra)
48         return d
49
50
51     _AGG = {"tol_f1": 0.30, "f1@0.5": 0.40, "f1@0.25": 0.50, "merge_rate": 0.10,
52             "tol_precision": 0.25, "tol_recall": 0.40, "precision": 0.35, "recall": 0.55,
53             "split_rate": 0.12, "segment_count_ratio": 1.2, "dstart_abs_median": 1.0,
```

```

54         "dend_abs_median": 2.0}
55
56     TOLS = nr.Tolerances() # defaults
57
58
59     # ----- #
60     # summarize_run
61     # ----- #
62     def test_summarize_run_extracts_means_and_per_video():
63         eval_result = {
64             "preset": dict(_PRESET), "iou": 0.5, "min_crossings": 0,
65             "rows": [{"name": "v1", "tol_f1": 0.3, "f1@0.5": 0.4, "f1@0.25": 0.5,
66 "merge_rate": 0.1,
67             "split_count": 2, "merge_count": 1, "num_pred": 9, "num_gt": 10}],
68             "aggregate": {"n": 1, "tol_f1": {"mean": 0.3}, "f1@0.5": {"mean": 0.4},
69                 "f1@0.25": {"mean": 0.5}, "merge_rate": {"mean": 0.1}},
70         }
71         s = nr.summarize_run(eval_result)
72         assert s["n"] == 1
73         assert s["aggregate"]["tol_f1"] == 0.3
74         assert s["aggregate"]["f1@0.5"] == 0.4
75         assert s["per_video"]["v1"]["split_count"] == 2
76         assert s["per_video"]["v1"]["merge_count"] == 1
77         # Missing aggregate key ? None, not a crash.
78         assert s["aggregate"]["precision"] is None
79
80     # ----- #
81     # compare_config ? clean / regression / improvement / within-tolerance
82     # ----- #
83     def test_identical_summaries_pass():
84         base = _summary(_AGG, {"v1": _pv(), "v2": _pv()})
85         cur = _summary(_AGG, {"v1": _pv(), "v2": _pv()})
86         cmp = nr.compare_config("c", base, cur, TOLS)
87         assert cmp.status == "ok"
88         assert cmp.regressions == []
89
90
91     def test_higher_better_drop_beyond_tol_is_regression():
92         base = _summary(_AGG, {"v1": _pv()})
93         cur = _summary(**_AGG, "tol_f1": 0.20}, {"v1": _pv()}) # 0.30 ? 0.20
94         cmp = nr.compare_config("c", base, cur, TOLS)
95         assert cmp.status == "regression"
96         regs = cmp.regressions
97         assert len(regs) == 1 and regs[0].metric == "tol_f1"
98         assert regs[0].delta == pytest.approx(-0.10)
99
100
101     def test_higher_better_drop_within_tol_is_clean():
102         base = _summary(_AGG, {"v1": _pv()})
103         cur = _summary(**_AGG, "tol_f1": 0.30 - 0.004}, {"v1": _pv()}) # under
metric_tol=0.005
104         cmp = nr.compare_config("c", base, cur, TOLS)
105         assert cmp.status == "ok" and cmp.regressions == []
106
107
108     def test_higher_better_rise_is_improvement_not_regression():
109         base = _summary(_AGG, {"v1": _pv()})
110         cur = _summary(**_AGG, "f1@0.5": 0.50}, {"v1": _pv()}) # 0.40 ? 0.50
111         cmp = nr.compare_config("c", base, cur, TOLS)
112         assert cmp.status == "ok"
113         assert [f.metric for f in cmp.improvements] == ["f1@0.5"]
114
115
116     def test_merge_rate_rise_is_regression():

```

```

117     base = _summary(_AGG, {"v1": _pv()})
118     cur = _summary(**_AGG, "merge_rate": 0.30}, {"v1": _pv()}) # lower-better, rose
119     cmp = nr.compare_config("c", base, cur, TOLS)
120     assert cmp.status == "regression"
121     assert cmp.regressions[0].metric == "merge_rate"
122
123
124 def test_merge_rate_drop_is_improvement():
125     base = _summary(_AGG, {"v1": _pv()})
126     cur = _summary(**_AGG, "merge_rate": 0.00}, {"v1": _pv()})
127     cmp = nr.compare_config("c", base, cur, TOLS)
128     assert cmp.status == "ok"
129     assert [f.metric for f in cmp.improvements] == ["merge_rate"]
130
131
132 # ----- #
133 # Per-video over-seg guard
134 # ----- #
135 def test_per_video_split_increase_is_regression():
136     base = _summary(_AGG, {"v1": _pv(split_count=2)})
137     cur = _summary(_AGG, {"v1": _pv(split_count=4)}) # 2 ? 4
138     cmp = nr.compare_config("c", base, cur, TOLS)
139     assert cmp.status == "regression"
140     r = cmp.regressions[0]
141     assert r.scope == "per_video" and r.metric == "split_count" and r.video == "v1"
142
143
144 def test_per_video_merge_increase_is_regression():
145     base = _summary(_AGG, {"v1": _pv(merge_count=0)})
146     cur = _summary(_AGG, {"v1": _pv(merge_count=2)})
147     cmp = nr.compare_config("c", base, cur, TOLS)
148     assert cmp.regressions[0].metric == "merge_count"
149
150
151 def test_per_video_decrease_is_not_regression():
152     base = _summary(_AGG, {"v1": _pv(split_count=5, merge_count=3)})
153     cur = _summary(_AGG, {"v1": _pv(split_count=2, merge_count=0)}) # improved
154     cmp = nr.compare_config("c", base, cur, TOLS)
155     assert cmp.status == "ok" and cmp.regressions == []
156
157
158 def test_over_seg_tol_allows_small_increase():
159     tols = nr.Tolerances(over_seg_tol=1)
160     base = _summary(_AGG, {"v1": _pv(split_count=2)})
161     cur = _summary(_AGG, {"v1": _pv(split_count=3)}) # +1, within tol
162     assert nr.compare_config("c", base, cur, tols).status == "ok"
163     cur2 = _summary(_AGG, {"v1": _pv(split_count=4)}) # +2, beyond tol
164     assert nr.compare_config("c", base, cur2, tols).status == "regression"
165
166
167 # ----- #
168 # Staleness: corpus change + config-definition change
169 # ----- #
170 def test_corpus_change_is_stale_not_regression():
171     base = _summary(_AGG, {"v1": _pv(), "v2": _pv()})
172     cur = _summary(_AGG, {"v1": _pv(), "v3": _pv()}) # v2 removed, v3 added
173     cmp = nr.compare_config("c", base, cur, TOLS)
174     assert cmp.status == "stale"
175     assert cmp.added_videos == ["v3"] and cmp.removed_videos == ["v2"]
176     assert cmp.aggregate_gated is False
177
178
179 def test_corpus_change_still_catches_intersection_regression():
180     # v1 is shared and its split_count regressed ? regression must win over stale.
181     base = _summary(_AGG, {"v1": _pv(split_count=2), "v2": _pv()})

```

```

182     cur = _summary(_AGG, {"v1": _pv(split_count=9), "v3": _pv()})
183     cmp = nr.compare_config("c", base, cur, TOLS)
184     assert cmp.status == "regression"
185     assert any(f.video == "v1" and f.metric == "split_count" for f in cmp.regressions)
186
187
188     def test_corpus_change_skips_aggregate_gate():
189         # Aggregate dropped, but corpus changed ? not comparable ? no aggregate regression.
190         base = _summary(**_AGG, "tol_f1": 0.50}, {"v1": _pv(), "v2": _pv()})
191         cur = _summary(**_AGG, "tol_f1": 0.10}, {"v1": _pv(), "v3": _pv()})
192         cmp = nr.compare_config("c", base, cur, TOLS)
193         assert cmp.status == "stale"
194         assert not any(f.scope == "aggregate" for f in cmp.regressions)
195
196
197     def test_config_definition_change_is_stale():
198         base = _summary(_AGG, {"v1": _pv()}, preset=dict(_PRESET))
199         changed = {**_PRESET, "max_window_duration": 6.0} # different windowing definition
200         cur = _summary(_AGG, {"v1": _pv()}, preset=changed)
201         cmp = nr.compare_config("c", base, cur, TOLS)
202         assert cmp.status == "stale" and cmp.aggregate_gated is False
203
204
205     def test_preset_name_difference_is_not_stale():
206         base = _summary(_AGG, {"v1": _pv()}, preset={**_PRESET, "name": "served"})
207         cur = _summary(_AGG, {"v1": _pv()}, preset={**_PRESET, "name": "milestone"})
208         cmp = nr.compare_config("c", base, cur, TOLS) # only the cosmetic name differs
209         assert cmp.status == "ok"
210
211
212     def test_min_crossings_change_is_stale():
213         base = _summary(_AGG, {"v1": _pv()}, min_crossings=0)
214         cur = _summary(_AGG, {"v1": _pv()}, min_crossings=1)
215         assert nr.compare_config("c", base, cur, TOLS).status == "stale"
216
217
218     # ----- #
219     # compare_all + exit codes
220     # ----- #
221     def _wrap(configs):
222         return {"configs": configs}
223
224
225     def test_compare_all_pass():
226         base = _wrap({"default": _summary(_AGG, {"v1": _pv()})},
227                       "milestone": _summary(_AGG, {"v1": _pv()})})
228         cur = _wrap({"default": _summary(_AGG, {"v1": _pv()})},
229                     "milestone": _summary(_AGG, {"v1": _pv()})})
230         res = nr.compare_all(base, cur, TOLS)
231         assert res.overall_status() == "PASS" and res.exit_code() == 0
232
233
234     def test_compare_all_regression_exit_1():
235         base = _wrap({"default": _summary(_AGG, {"v1": _pv()})})
236         cur = _wrap({"default": _summary(**_AGG, "tol_f1": 0.0}, {"v1": _pv()})})
237         res = nr.compare_all(base, cur, TOLS)
238         assert res.has_regression and res.exit_code() == 1 and res.overall_status() ==
"REGRESSION"
239
240
241     def test_compare_all_stale_exit_4():
242         base = _wrap({"default": _summary(_AGG, {"v1": _pv(), "v2": _pv()})})
243         cur = _wrap({"default": _summary(_AGG, {"v1": _pv()})}) # v2 dropped
244         res = nr.compare_all(base, cur, TOLS)
245         assert res.has_stale and not res.has_regression and res.exit_code() == 4

```

```

246
247
248 def test_regression_takes_precedence_over_stale():
249     # One config stale, another regressed ? overall REGRESSION / exit 1.
250     base = _wrap({"a": _summary(_AGG, {"v1": _pv(), "v2": _pv()}),
251                  "b": _summary(_AGG, {"v1": _pv()})})
252     cur = _wrap({"a": _summary(_AGG, {"v1": _pv()}),                                # stale (v2
gone)
253                  "b": _summary(_AGG, {"tol_f1": 0.0}, {"v1": _pv()})}) # regressed
254     res = nr.compare_all(base, cur, TOLS)
255     assert res.has_regression and res.has_stale and res.exit_code() == 1
256
257
258 def test_config_missing_in_baseline_is_stale():
259     base = _wrap({"default": _summary(_AGG, {"v1": _pv()})})
260     cur = _wrap({"default": _summary(_AGG, {"v1": _pv()}),
261                  "milestone": _summary(_AGG, {"v1": _pv()})}) # new config
262     res = nr.compare_all(base, cur, TOLS)
263     assert res.has_stale and res.exit_code() == 4
264
265
266 # ----- #
267 # tracked_configs ? the DEFAULT vs MILESTONE definitions
268 # ----- #
269 def test_tracked_configs_default_tracks_served_post_r1_flip():
270     # eval?serve discipline: the nightly "default" config IS the SERVED preset, so it
always
271     # mirrors what production ships. After the R1 milestone flip (cap=6 + R4 now
default-ON, PR
272     # #204), the served operating point IS the milestone ? so "default" now carries
those knobs
273     # and equals "milestone". (The nightly's own config-drift detector flags the frozen
baseline
274     # STALE on the first post-flip run ? re-baseline with --update-baseline; see the R1
launch report.)
275     cfigs = {c.name: c for c in nr.tracked_configs()}
276     d = cfigs["default"].preset
277     assert d.max_window_duration == 6.0 and d.inactivity_end == 1.5 and d.hard_cap is
False
278     assert cfigs["default"].min_crossings == 0 # W2 net-crossings stays OFF
279     # default now mirrors the (explicit) milestone config ? production shipped the
milestone.
280     assert d.max_window_duration == cfigs["milestone"].preset.max_window_duration
281     assert d.inactivity_end == cfigs["milestone"].preset.inactivity_end
282
283
284 def test_tracked_configs_milestone_is_cap6_plus_r4():
285     cfigs = {c.name: c for c in nr.tracked_configs()}
286     m = cfigs["milestone"].preset
287     assert m.max_window_duration == 6.0
288     assert m.inactivity_end == 1.5
289     assert m.inactivity_rally_thresh == pytest.approx(0.20)
290     assert m.inactivity_min_density == pytest.approx(0.30)
291     # milestone shares the SERVED base knobs (single-sourced), only adds R3/R4.
292     assert m.velocity_thresh == cfigs["default"].preset.velocity_thresh
293
294
295 def test_tracked_configs_baseline_off_is_the_preflip_floor():
296     # The pinned all-OFF reference (the pre-R1-flip windowing) ? keeps the milestone's
value visible
297     # in every nightly report even though DEFAULT is now the milestone (post-#204).
298     cfigs = {c.name: c for c in nr.tracked_configs()}
299     assert "baseline_off" in cfigs
300     b = cfigs["baseline_off"].preset
301     # every quality knob forced OFF

```

```

302     assert b.max_window_duration == 0.0 and b.inactivity_end == 0.0
303     assert b.hard_cap is False and b.blob_fallback is False
304     assert cfgs["baseline_off"].min_crossings == 0          # W2 net-crossings off too
305     # but anchored on the SAME SERVED base knobs, so default?baseline_off is a clean A/B
306     assert b.velocity_thresh == cfgs["default"].preset.velocity_thresh
307     assert b.merge_gap == cfgs["default"].preset.merge_gap
308     assert b.min_window_duration == cfgs["default"].preset.min_window_duration
309     # and it really is OFF where the shipped default is now ON (the floor ? the
milestone)
310     assert b.max_window_duration != cfgs["milestone"].preset.max_window_duration
311
312
313     # ----- #
314     # Report formatting (smoke)
315     # ----- #
316     def test_format_report_has_status_and_regressions():
317         base = _wrap({"default": _summary(_AGG, {"v1": _pv()})})
318         cur = _wrap({"default": _summary(**_AGG, "tol_f1": 0.0}, {"v1":
_pv(split_count=9)}))
319         base["git_commit"] = "aaa"
320         base["generated_at"] = "2026-06-16"
321         cur["git_commit"] = "bbb"
322         cur["manifest"] = "m.json"
323         cur["iou"] = 0.5
324         res = nr.compare_all(base, cur, TOLS)
325         report = nr.format_report(base, cur, res, TOLS, "2026-06-17")
326         assert "Nightly rally-quality regression ? 2026-06-17" in report
327         assert "REGRESSION" in report
328         assert "tol_f1" in report
329         assert "Config `default`" in report
330
331
332     def test_format_report_pass_badge():
333         base = _wrap({"default": _summary(_AGG, {"v1": _pv()})})
334         cur = _wrap({"default": _summary(_AGG, {"v1": _pv()})})
335         res = nr.compare_all(base, cur, TOLS)
336         report = nr.format_report(base, cur, res, TOLS, "2026-06-17")
337         assert "PASS" in report
338
339
340     def test_tolerances_defaults():
341         t = nr.Tolerances()
342         assert t.metric_tol == 0.005 and t.rate_tol == 0.010 and t.over_seg_tol == 0
343         assert t.to_dict()["metric_tol"] == 0.005
344

```

39. assistant (2026-06-22T09:09:20.365Z)

Perfect! Now I have a comprehensive understanding. Let me look at how the pipeline's end-to-end flow works to understand where reel-count data would come from:

40. user (2026-06-22T09:09:22.190Z)

```

./backend/config/models.py
./backend/main.py
./backend/pipeline/stitcher/compiler.py
./backend/reporting/harvest.py
./backend/tools/export.py

```

41. user (2026-06-22T09:09:27.251Z)

```

1     """Minimal Value Layer exports (first slice of review item 3).
2
3     Provides timeline exports (EDL, JSON, CSV) and basic stats over the

```

```

4     play_segments returned by the indexer (or query_play_segments). These are
5     the "time-saved editing is table stakes" deliverables and also give a
6     machine-readable foundation for later human correction / golden feeding
7     and clip galleries.
8
9     Kept deliberately small and pure (no DB, no side effects) so they are
10    easy to call from CLI, notebooks, or a future API endpoint.
11    """
12
13    from __future__ import annotations
14
15    import json
16    from typing import Any, Dict, Iterable, List, Optional
17
18
19    def _to_list(segments: Iterable[Dict[str, Any]]) -> List[Dict[str, Any]]:
20        return list(segments) if not isinstance(segments, list) else segments
21
22
23    def _segment_metadata(s: Dict[str, Any]) -> Dict[str, Any]:
24        """A play segment's metadata as a dict, whether stored inline (segmenter output)
25        or as a JSON string (DB round-trip via query_play_segments)."""
26        md = s.get("metadata") or {}
27        if isinstance(md, str):
28            try:
29                md = json.loads(md)
30            except Exception:
31                md = {}
32        return md if isinstance(md, dict) else {}
33
34
35    def _fmt_hms(seconds: float) -> str:
36        """HH:MM:SS, matching the segmenters' _fmt_ts so the summary block lines up with
37        the per-rally [rally] log lines and the existing timestamp style."""
38        total = max(0, int(float(seconds)))
39        h, rem = divmod(total, 3600)
40        m, s = divmod(rem, 60)
41        return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44    def format_rally_summary(
45        segments: Iterable[Dict[str, Any]], title: str = "Rally summary"
46    ) -> str:
47        """Render a compact, scannable end-of-run block for manual QA against the rally CSV.
48
49        One header line (total rallies + total highlight duration in HH:MM:SS) followed by a
50        per-rally table ``# start end dur(s) shot_count`` in time order. Pure (no DB, no
51        I/O):
52
53        the same play_segment dicts that feed ``segments_to_csv`` / the rally CSV go in and
54        text comes out, so the log and CSV line up row-for-row for eyeball verification.
55
56        A segment with no ``shot_count`` in its metadata renders the count as ``-``.
57        """
58        segs = _to_list(segments)
59        total_dur = sum(
60            float(s.get("duration", float(s.get("end_time", 0)) - float(s.get("start_time",
61            0))))
62            for s in segs
63        )
64        header = (
65            f"=== {title}: {len(segs)} rallies, "
66            f"{_fmt_hms(total_dur)} ({total_dur:.1f}s) total play ==="
67        )
68        if not segs:
69            return header + "\n (no rallies detected)"

```



```

67
68     lines = [header, f"{'#':>3} {'start':<8} {'end':<8} {'dur(s)':>7} {'shots':>6}"]
69     for i, s in enumerate(segs, start=1):
70         start = float(s.get("start_time", 0.0))
71         end = float(s.get("end_time", start))
72         dur = float(s.get("duration", end - start))
73         shot = _segment_metadata(s).get("shot_count")
74         shot_str = "-" if shot is None else str(shot)
75         lines.append(
76             f"{i:>3} {_fmt_hms(start):<8} {_fmt_hms(end):<8} {dur:>7.1f}
{shot_str:>6}"
77         )
78     return "\n".join(lines)
79
80
81     def basic_stats(segments: Iterable[Dict[str, Any]]) -> Dict[str, Any]:
82         """Return simple aggregate stats for a list of play segments.
83
84         Keys: count, total_duration_s, avg_duration_s, min_duration_s, max_duration_s.
85         All durations are in seconds (float).
86         """
87         segs = _to_list(segments)
88         if not segs:
89             return {
90                 "count": 0,
91                 "total_duration_s": 0.0,
92                 "avg_duration_s": 0.0,
93                 "min_duration_s": 0.0,
94                 "max_duration_s": 0.0,
95             }
96         durs = [float(s.get("duration", (s.get("end_time", 0) - s.get("start_time", 0))))
for s in segs]
97         total = sum(durs)
98         return {
99             "count": len(segs),
100             "total_duration_s": round(total, 2),

```

42. assistant (2026-06-22T09:09:30.034Z)

Perfect! Now I have a clear picture. Let me create a comprehensive structured response:

43. user (2026-06-22T09:10:36.466Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 11593 bytes): {"summary": "Design for a NIGHTLY FULL-PIPELINE REEL-COUNT REGRESSION TEST that reuses the established `scripts/nightly\_regression.py` infrastructure. The new test will run the configured pipeline end
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

44. user (2026-06-22T09:11:03.419Z)

Structured output provided successfully

45. assistant (2026-06-22T09:11:34.945Z)

Perfect. Now let me provide a human-readable summary as well:

Nightly Full-Pipeline Reel-Count Regression Design

I've analyzed the badminton-highlight-indexer codebase and mapped the design for extending the established nightly regression infrastructure to include **full-pipeline reel-count assertions**.

Current Nightly Infrastructure (Established in PR #202)

The existing ``scripts/nightly_regression.py`` tripwire re-scores a frozen golden corpus against

`**cached WASB trajectories**` using three tracked windowing configs:

- `**DEFAULT**` ? SERVED production windowing (now == R1 milestone post-flip)
- `**MILESTONE**` ? explicit R3+R4 windowing (cap=6 + inactivity trim)
- `**BASELINE_OFF**` ? pre-R1 all-OFF floor (historical reference)

`**Pattern:**`

1. Manifest-driven (``eval_baselines/fixtures/manifest.json`` or local ``output/human_lovo_manifest.json``)
2. Frozen baseline snapshot (``eval_baselines/nightly_baseline.json``)
3. Per-config + per-video metrics in ``PV_SNAPSHOT`` and ``AGG_SNAPSHOT`` tuples
4. Gated metrics: F1-like (higher-better), `merge_rate` (lower-better), per-video over-seg guards (split/merge counts)
5. Output: dated markdown + JSON reports (``output/nightly_regression/{date}.md|.json``), exit codes 0/1/4
6. Windows Scheduled Task wrapper (runs nightly at 03:00, gracefully no-ops if corpus missing)

Reel-Count Integration Points

`**Where reel data originates:**`

- ``backend/main.py::compile_highlights()`` (line 706) ? calls ``VideoCompiler.compile_highlights(local_src, time_pairs, out_name)``
- ``backend/pipeline/stitcher/compiler.py::compile_highlights()`` (line 305) ? takes ``List[Tuple[float, float]]`` segments, merges overlaps, returns output path
- Input segments come from: ``db.query_play_segments(video_id, {})`` (segmentation output)
- Counting: ``backend/tools/export.py::basic_stats()`` (line 81) does ``len(segs)`` ? use this or extract directly

`**Snapshot structure**` (from ``eval_baselines/nightly_baseline.json``):

```
...
{
  "configs": {
    "default": {
      "aggregate": {...},
      "per_video": {
        "video_name": {
          "tol_f1", "f1@0.5", "merge_rate", "split_count", "merge_count", ...
        }
      },
      "n": 15 // number of videos in corpus
    }
  },
  "generated_at": "2026-06-19",
  "git_commit": "a240221"
}
...
```

Recommended Design: Extend ``nightly_regression.py`` (vs Sibling Script)

`**Why extend, not create `scripts/nightly_reelcount.py`:**`

1. Reuses manifest loading, baseline I/O, config management, report formatting
2. One-line JSON edit to add videos (same pattern as quality metrics)
3. Single dated report covers both quality + reel stability

`**Implementation outline:**`

1. `**Add reel-count fields to snapshots**` (lines 88-96):
 - ``PV_SNAPSHOT += ("reel_count",)`` ? segments produced per video
 - ``AGG_SNAPSHOT += ("reel_count_mean",)`` ? average per-video reel count
2. `**Extend `run_corpus()`**` (lines 493-522) to process end-to-end:
 - Load manifest video entries
 - For each video + config: call `segmenter` (`process_video`) ? query segments ? compile ? count

output clips

- Store reel_count in per-video metrics
- *Or* run a lightweight wrapper that reuses `VideoCompiler` + your existing processing pipeline

3. ****Add gating logic**** (lines 249-314):
 - Add `reel\_count` to `GATED\_HIGHER` (expect it stable or improve; catch drops)
 - Per-video reel_count regression: flag if `current < baseline - tolerance`
 - Aggregate reel_count_mean: drop beyond threshold = regression
4. ****Extend report formatting**** (lines 368-460):
 - Add reel_count column to per-video table (alongside split/merge counts)
 - Show aggregate reel_count delta in the config summary table
5. ****Baseline snapshot**** (lines 532-545):
 - `save\_baseline()` captures reel_count values alongside quality metrics
 - Baseline comparison in `compare\_all()` gates on reel_count like F1

****For extensibility ? one-line manifest edit:****

The existing `eval\_baselines/fixtures/manifest.json` has schema `v1` (line 13) with arbitrary metadata fields. Adding a new golden video:

```
```json
{
 "name": "new_golden_short_1",
 "fps": 30.0,
 "paths": { ... },
 "slug": "new_golden_short_1"
}
```
```

Just append to the manifest array. The nightly script auto-discovers.

Critical Constraints & Trade-offs

****Performance:**** Full-pipeline run is ****expensive**** (segmentation hits GPU/Gemini + stitching encodes video). Current nightly re-scores cached trajectories (CPU, ~seconds). Full run = ****10-60 min per video****.

- ****Option A:**** Use 1-2 short synthetic test videos (< 1 min each) to keep nightly under time budget
- ****Option B:**** Run full-corpus test as a separate on-demand task (not nightly)

****Determinism:**** Reel counts must be stable across re-runs. Confirmed for WASB native stream path (2026-06-20 validation), but needs owner confirmation on their machine.

****Baseline bloat:**** Current `nightly\_baseline.json` ~100 KB. Adding full segment lists per video could balloon to MBs. Recommend storing only the COUNT, not full segment metadata.

****Manifest points to local files:**** Golden videos live outside the repo on the owner's machine (not cached). Manifest must have valid file paths or fetched URIs.

****Deliverables:**** Extend `scripts/nightly\_regression.py` with reel-count tracking; update tracked_configs to define video set; snapshot new baseline with `--update-baseline`; wrappers unchanged.